Settling 650 unsolved BOBILib instances: 531 published optima, 79 new optimality proofs and 40 infeasibility certificates

Brian J. Connolly*

Beehive Industries

ABSTRACT

The Bilevel Optimization (Benchmark) Instance Library (BOBILib) holds 2,654 mixed-integer bilevel linear problems, and its metadata, last modified on 27 May 2026 and downloaded on 18 October 2026, lists 1,594 of them as feasible or open (here: unsolved). This paper asks which of these instances can be settled, that is, proven optimal or proven to have no bilevel feasible point. Most share structure that general solvers do not use: 1,254 are packing interdiction games, 60 are clique interdiction problems dense enough for extremal graph theory to apply, and most of the rest are built from models of the Mixed Integer Programming Library (MIPLIB), whose published optima serve as lower bounds. Published work had already solved 531 of the 1,594. Methods built for these structures prove 79 more optimal, among them all 16 knapsack interdiction instances open in the literature and 57 of the 60 clique instances, and show that 40 have no bilevel feasible point. Of these 119 results, 113 have exact certificates; the other 6 rest on published MIPLIB optima. The work also gives a verified point for 228 instances where BOBILib lists none, 167 of them the first known, and improves on every published value for 474 instances. In all, 650 instances (41%) are now settled. Every current solution file is in BOBILib's format and passes a tight test of follower optimality. The knapsack interdiction optima are computational proofs supported by lemmas proved in the Lean theorem prover. An updated status file, the solution files and the certificates are released as a data package.

Keywords: bilevel optimization, interdiction games, knapsack interdiction, clique interdiction, benchmark library, optimality certificates, formal verification

1. INTRODUCTION

In a bilevel optimization problem a leader decides first and a follower then solves its own optimization problem, knowing the leader's decision. The leader's cost depends on the follower's answer, so the leader has to anticipate it (Figure 1). This is the leader-follower game of Stackelberg,[1] and it is the natural model of attack and defense: an interdictor removes or degrades parts of a system within a budget, and an operator then makes the best use of what is left.[2,3] Mixed-integer bilevel linear problems (MIBLPs), in which both levels are mixed-integer linear programs, are hard in theory and in practice. They are Σ_2^p-hard in general,[4] and even the bilevel knapsack problem with interdiction constraints is Σ_2^p-complete.[5] A textbook and a recent survey cover the field.[6,7]

Benchmark libraries focus research effort. For single-level mixed-integer programming, the Mixed Integer Programming Library (MIPLIB)[8–10] records for every instance whether it is solved, and its open instances have long been targets for new methods. BOBILib, the Bilevel Optimization (Benchmark) Instance Library of Thürauf, Kleinert, Ljubić, Ralphs and Schmidt,[11,12] plays the same role for MIBLPs. It collects 2,654 instances, most of them from the literature, in one file format, with solution files, a feasibility checker and a status per instance. Its metadata (last modified on 27 May 2026, downloaded on 18 October 2026) lists 1,027 instances as optimal, 33 as infeasible, 1,175 as feasible (a point is known, but not proven optimal) and 419 as open (no point is known). The statuses come from runs of two general-purpose bilevel solvers, MibS[13,14] and the solver of Fischetti et al. in its setting MIX++, or HC++ where MIX++ does not apply,[15,16] each for one hour on one thread. They were fixed when the instances were collected. Some of the specialized results discussed below appeared later, and a few earlier ones, from MIX++ runs with four threads or a more aggressive setting, were not carried over (Section 2.9).

This paper asks a simple question: which of the 1,594 feasible or open instances (below, *unsolved*) can be *settled*, that is, proven optimal or proven to have no bilevel feasible point, and are there methods that settle many of them at once? The goals of the work were to:

*Further author information: E-mail: brian.connolly@beehiveind.com

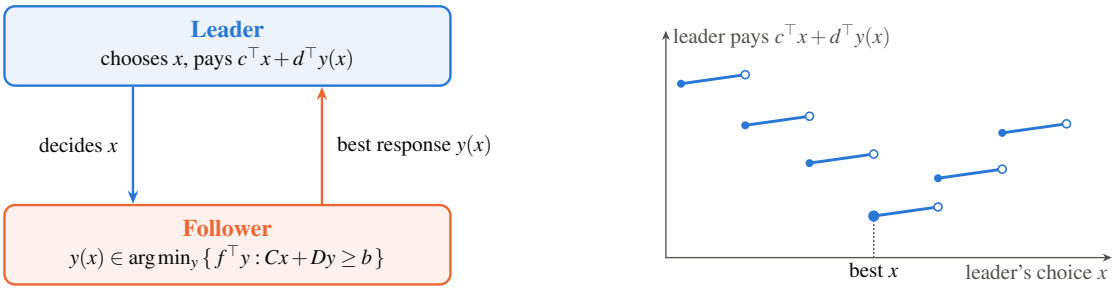


Figure 1. The bilevel game. The leader chooses x and pays $c^\top x + d^\top y(x)$, where $y(x)$ is the follower's best response to x. The leader's cost jumps wherever the follower's answer changes (right: schematic).

- reconcile BOBILib's statuses with the per-instance results published since the instances were collected;
- find structure shared by many of the remaining instances, and exploit it with methods that settle whole families rather than single instances;
- make every claim checkable by a reader who does not trust my code: verified solution files in BOBILib's own format, a certificate for every bound with its degree of rigor stated, and machine-checked proofs where the argument is new.

The main findings are these (Figure 2).

- Published work had already solved 531 of the 1,594 instances: 428 with methods built for knapsack interdiction,[17–20] 96 with the decision diagrams of Lozano, Bergman and Cire[21] and 7 by other methods (Section 2.9). Reconciling the statuses with these results settles more instances than any method of this paper.
- A new exact search proves all 16 knapsack interdiction instances that were open in the literature optimal, the slowest in 10.0 seconds. In all of them the optimum equals the trivial upper bound, so the whole difficulty lies in the lower bound. These are computational proofs supported by formally verified lemmas: the main pruning lemmas are proved in Lean, and two integer-only provers repeat the search.
- Two theorems of extremal graph theory, Mantel's and Brouwer's, together with partition certificates, prove 57 of the 60 edge clique interdiction instances optimal, at values 3 and 4 where BOBILib lists 5 to 13. Brouwer's theorem applies once every split of the vertices into three parts has more edges inside the parts than the leader may delete; checking this reduces to 170 finite cases, each with a branch-and-bound tree that an integer checker of 174 lines of code verifies.
- In 40 instances the follower's problem is infeasible or unbounded for every leader choice, so they have no bilevel feasible point at all; all 40 certificates are checked in exact rational arithmetic. A verified point that meets the published MIPLIB optimum of the high point relaxation (HPR, the problem without the follower's optimality condition; Section 2.1) makes 6 MIPLIB-derived instances optimal by the accepted conventions of MIPLIB and BOBILib (solver-certified, not exact), 4 of them open before. Another 228 instances get a verified point where BOBILib lists none, 167 of them the first point anywhere, and 474 instances of the OR set (Section 2.3) get a point better than every published one.
- In total 650 (41%) of the 1,594 instances are settled: 531 by published work and 119 (7.5%) by this work (79 optimal and 40 without a bilevel feasible point), so 610 are optimal. 113 of the 119 have exact certificates. All 1,478 current solution files pass a tight test of follower optimality, so no point relies on BOBILib's tolerance. The results are released with an updated status file in MIPLIB's `.solu` format.

What is new, and what is reused. New are the reconciliation itself, the robust subset-sum search with tables rebuilt at every node, the use of Brouwer's stability theorem for edge clique interdiction with checkable certificates for its finite cases, the Lean proofs, and the verified data package, in which every settled instance and every new point carries a certificate class. The other arguments are known and are applied here systematically: dualizing the follower's linear programming relaxation

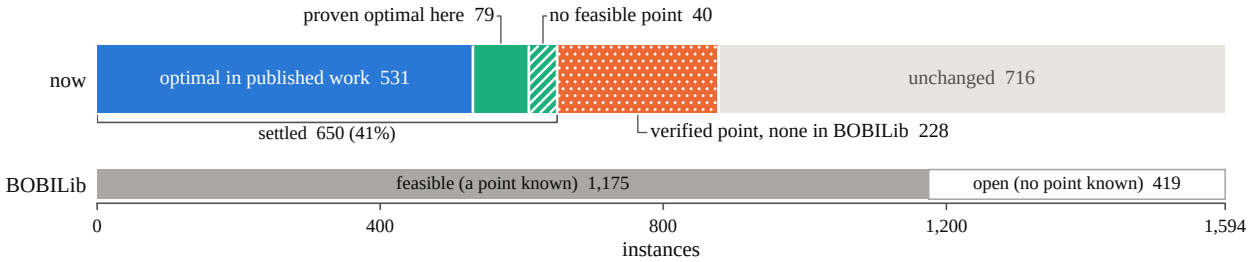


Figure 2. Where BOBILib's 1,594 feasible or open instances stand. Of these, 650 (41%) are settled: 531 were already optimal in published work, 79 are proven optimal here and 40 have no bilevel feasible point. Another 228 get a verified point where BOBILib lists none (for 167 of them the first point anywhere) and 716 are unchanged. The thin bar shows BOBILib's own split (1,175 feasible, 419 open).

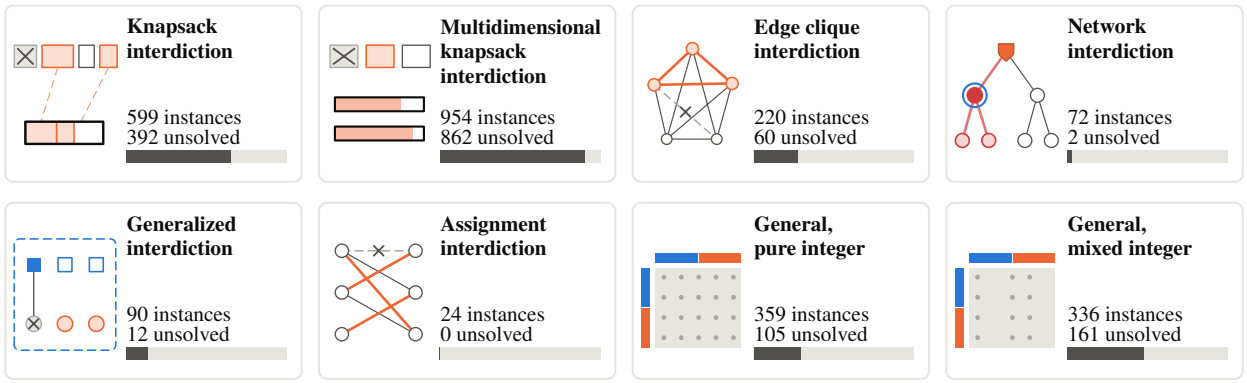


Figure 3. BOBILib at a glance: one tile per family, with its number of instances and the number BOBILib lists as feasible or open (unsolved; the bar shows their share). In the glyphs, blue marks the leader, orange the follower, and grey with a cross what the leader blocks. In all, 2,654 instances, 1,594 of them unsolved; BOBILib's other 33 non-optimal instances are proven infeasible and all belong to the general families.

into one mixed-integer program[17,22,23] and the integrality gap of a packing problem with few rows, which follows from the fact that a basic LP solution has at most as many fractional variables as rows,[24] a bracket that Weninger's 2026 preprint also states for one follower knapsack;[25] the online lower bound of Weninger and Fukasawa, whose code already plays it on counted copies of identical items;[19,26] the one-LP test of Fischetti et al. for followers that are unbounded for every leader choice, which extends a lemma of Xu and Wang;[27,28] Meyer's argument that a rational mixed-integer program that is feasible and bounded attains its optimum;[29] Turán-type counts of the deletions that leave no clique of a given size, which exact methods for edge clique interdiction already apply within single cliques;[30,31] the bound HPR $\leq$ OPT of the high point relaxation;[32] and exhaustive enumeration.

Section 2 describes BOBILib's problem families for readers outside the field and says what published work had solved. Section 3 describes how each claim is checked and the methods, Section 4 gives the computational setup and the results, and Section 5 discusses what a benchmark library could do with them, the limitations of this work and what stays hard. The proofs, the verification protocol and the per-instance tables are in a separate supplementary document; its sections are cited as Supp. Sec. A, B, and so on, and its results as, for example, Supp. Lemma B.1.

2. THE PROBLEMS

BOBILib's 2,654 instances fall into eight families (Figure 3). Six are interdiction problems: a leader, the interdictor, first blocks parts of a follower's optimization problem within a budget, and the follower then makes the best of what is left. The other two families hold general bilevel problems, most of them built from single-level mixed-integer linear programs (MILPs) of MIPLIB. This section describes each family: a short story, the formal model, how BOBILib's instances are generated and named, their sizes (Table 1), what makes them hard, and what was known; Table 2 anticipates the outcome of this work. The stories only illustrate the models: the instances themselves are random, or derived from benchmark problems. In the instance names below, $\langle n \rangle$ stands for a number, for example the number of items.

Table 1. Sizes of BOBILib's families: the sets (BOBILib's tags), the number of instances, how many BOBILib lists as feasible or open (unsolved), and the ranges of the numbers of leader and follower variables and rows (rows without the objective; BOBILib's metadata, checked against the instance files). The general families contain MIPLIB models of every size.

					Variables		Rows	
Family	Sets	Instances	Unsolved	Size	Leader	Follower	Leader	Follower
Knapsack interdiction	`kp, cclw, inter-kp`	599	392	10–500 items	10–500	10–500	1	11–501
Multidim. knapsack interdiction	`or, imkp`	954	862	10–500 items	10–500	10–500	1–29	11–529
Edge clique interdiction	`clique, bcpins, plusbcpins`	220	60	8–60 vertices	19–1,593	8–1,653	1	28–3,363
Network interdiction	`inter-fire`	72	2	22–79 nodes	22–79	44–158	1	41–974
Generalized interdiction	`generalized`	90	12	$n_x, n_y \in \{20, 30\}$	40–50	40–50	20	30–50
Assignment interdiction	`inter-assig`	24	0	5×5	25	25	1	45
General, pure integer	`miplib2010/2017, miplib3, denegre, zhang, misc`	359	105	MILPs	1–714,549	1–714,549	0–84,788	3–169,576
General, mixed integer	`miplib2010/2017, xuwang, xularge`	336	161	MILPs	10–399,808	10–399,808	0–480,585	4–961,170

2.1 Bilevel Problems in BOBILib Form

BOBILib writes every instance with a minimizing leader; as in mathematical programming, a *row* is a linear constraint:

$$\min_{x,y} \; c^\top x + d^\top y \quad \text{s.t.} \quad Ax + By \geq a, \qquad y \in \arg\min_{y'} \big\{ f^\top y' : Cx + Dy' \geq b \big\}, \tag{1}$$

with integrality requirements on some of the leader's variables x and the follower's variables y. For a fixed x the follower solves $P(x) = \min\{f^\top y : Cx + Dy \geq b\}$ (with its integrality requirements); x enters it only through the right-hand side. A point (x, y) is *bilevel feasible* if it satisfies the leader's rows and integrality requirements and y is optimal for $P(x)$, so checking a single point already means solving the follower's problem to optimality. When the follower has several optimal answers, the one best for the leader is assumed (the optimistic convention, which BOBILib uses throughout). An upper-level row with a nonzero coefficient on a follower variable is a *coupling row*; a leader variable that appears in the follower's rows is a *linking variable*. The *high point relaxation* (HPR) drops the follower's optimality and keeps every row of both levels,

$$\text{HPR} = \min\big\{ c^\top x + d^\top y : Ax + By \geq a,\ Cx + Dy \geq b,\ \text{integrality} \big\},$$

so HPR $\leq$ OPT, the bilevel optimum.

Each instance is a pair of files.[11] The MPS file is the HPR: every variable, every row of both levels, and the leader's objective. The AUX file lists the follower's variables with their objective coefficients f, and the follower's rows; every other variable and row belongs to the leader. In the interdiction families the leader's objective and the follower's are opposite ($c = 0$ and $f = -d$; for the generalized and assignment families, see below), so the game is zero-sum, the optimistic and pessimistic readings agree, and the model reads more naturally as $\min_x \max_y$. In most of them, each item j that the leader can block has a leader variable $x_j \in \{0, 1\}$ and a follower row $x_j + y_j \leq 1$ (an *interdiction row*): blocking item j takes it away from the follower.

2.2 Knapsack Interdiction

The story. A smuggler fills a container of limited capacity with the most valuable goods it can get. Before that, an inspector, with a limited budget of inspection hours, bans some of the goods; each ban costs hours. The inspector wants the smuggler's best load to be worth as little as possible. The inspector is the leader and the smuggler the follower.

The model. This is a packing interdiction problem. Items are $N = \{1, \ldots, n\}$. The leader chooses $x \in \{0, 1\}^n$ within budget rows $Gx \leq g$, and the follower packs what is left:

$$\Phi(x) = \max\big\{ p^\top y : Wy \leq c,\ y \leq \mathbf{1} - x,\ y \in \{0, 1\}^n \big\}, \qquad \text{OPT} = \min\big\{ \Phi(x) : Gx \leq g,\ x \in \{0, 1\}^n \big\}, \tag{2}$$

with nonnegative integer profits p, weights W and capacities c (in Sections 2.2, 2.3, 3.2 and 3.3, c denotes capacities, not the cost vector of (1)). The knapsack interdiction problem (KIP) has one leader row, $\gamma^\top x \leq \beta$ with costs γ and budget β, and one follower row, $w^\top y \leq c$ with weights w and capacity c; the multidimensional KIP (MKIP, Section 2.3) has several

Table 2. BOBILib's families and the outcome for the instances it lists as feasible or open. Published: optimal in published work (for 82 of them only the published value is used; Section 3.1). Here: proven optimal in this work. No point: no bilevel feasible point exists. New point: a verified bilevel feasible point where BOBILib lists none (for 61 of them a point had been published; Section 4.2).

		Feasible	Outcome for the feasible or open instances				
Family	Instances	or open	Published	Here	No point	New point	Unchanged
Knapsack interdiction	599	392	376	16	0	0	0
Multidim. knapsack interdiction	954	862	140†	0	0	140	582
Edge clique interdiction	220	60	0	57	0	0	3
Network interdiction	72	2	2	0	0	0	0
Generalized interdiction	90	12	4	0	0	0	8
Assignment interdiction	24	0	0	0	0	0	0
General, pure integer	359	105	9	0	6	14	76
General, mixed integer	336	161	0	6	34	74	47
Total	2,654	1,594	531	79	40	228	716

†Includes OR10x100-0.75_4.dat-0, whose published lower bound is inconsistent but whose published branch-and-cut bound closes it (Section 4.4).

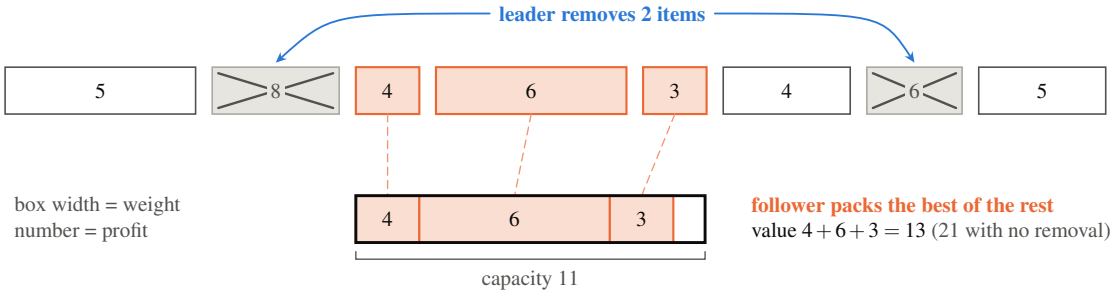


Figure 4. Knapsack interdiction on a toy instance (box width = weight, number = profit). The leader removes two items, and the follower packs the best of what is left into its knapsack. The removal shown cuts the follower's best packing from 21 to 13, and no other pair of removals does as well.

of either. In BOBILib form the leader minimizes $p^\top y$ and the follower minimizes $-p^\top y$. In the files the rows are the budget, the capacity and the n interdiction rows $x_j + y_j \le 1$, so an instance with n items has $2n$ variables and $n+2$ rows. In the *subset-sum class* each item's profit equals its weight ($p = w$), so the follower's value is the largest sum of surviving weights that fits the capacity. Figure 4 shows a small instance.

The instances. BOBILib has three KIP sets.

- `kp` (450 instances; Fischetti, Monaci and Sinnl[33]), named interKP-⟨n⟩-⟨R⟩-⟨t⟩-⟨I⟩, for example interKP-200-100-6-3: $n \in \{100, 200, \ldots, 500\}$ items, coefficient range $R = 100$, item class $t \in \{1, \ldots, 9\}$ and instance number $I \in \{1, \ldots, 10\}$. The ten instances with the same n and t share one set of items and differ only in I, which sets the capacity and with it the budget. The follower's weights and profits come from the knapsack generator of Martello, Pisinger and Toth[34] in class t: 1 uncorrelated, 2 weakly correlated, 3 strongly correlated, 4 inverse strongly correlated, 5 almost strongly correlated, 6 subset sum ($p = w$), 7 even-odd subset sum ($p = w$, even weights), 8 even-odd strongly correlated, 9 uncorrelated with similar weights. The instance number sets the capacity, $c = \lfloor I \sum_j w_j / 11 \rfloor$ (in classes 7 and 8 raised to the next odd number when even), so it grows from about 9% to 91% of the total weight. The leader's costs are integers between 0 and R, and its budget β is set within 10 of c, as in Caprara et al.'s instances.[35] In class 9 the budget exceeds the sum of all leader costs, so the leader can block every item; Fischetti et al. left these instances out as trivial, BOBILib keeps them.

- `cclw` (50 instances; Caprara, Carvalho, Lodi and Woeginger[17]), named interdiction⟨n⟩-⟨I⟩: the same construction with uncorrelated items between 1 and 100, $n \in \{35, 40, \ldots, 55\}$; here the leader's costs are drawn anew for each I.

- `inter-kp` (99 instances; DeNegre[36]), named K50⟨n⟩W⟨i⟩.KNP: $n \in \{10, 20, \ldots, 50\}$ items, instance $i \in \{1, \ldots, 20\}$ (K5010W01 is missing), built from bicriteria knapsack instances, whose first objective gives the follower's profits

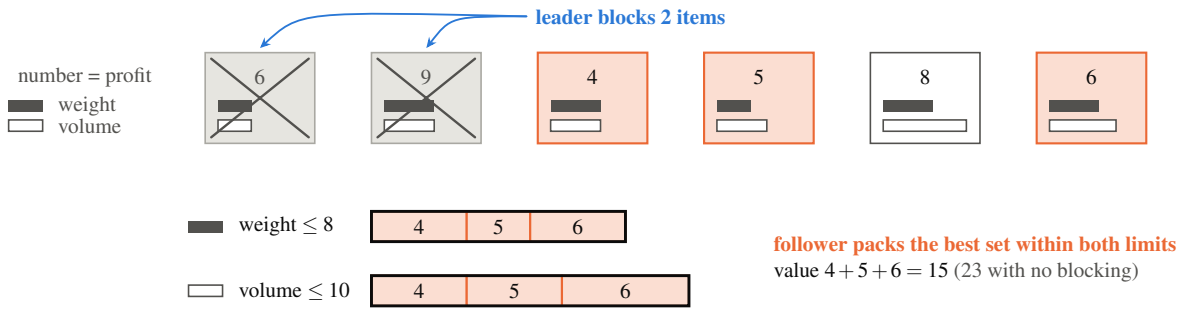


Figure 5. Multidimensional knapsack interdiction on a toy instance: every item has a profit, a weight and a volume, and the follower's knapsack has a limit for each. The leader blocks two items; the follower then packs the most valuable set that fits both limits (value 15 instead of 23). Blocking the two most valuable items (9 and 8) would leave 17.

and whose second gives the leader's costs; the budget is half the total cost, rounded up.

What makes it hard. The follower's problem alone, one knapsack with at most 500 items, is easy in practice. The difficulty is the leader's: it chooses among exponentially many subsets, and the follower's value $\Phi(x)$ jumps as x changes, with no convexity to exploit. Already this two-level knapsack problem is Σ_2^p-complete.[5] The HPR, the starting point of general bilevel solvers, is worthless here: it lets the follower pack nothing ($y = 0$), so its value is 0. The subset-sum classes are hard in another way: the follower can usually fill its knapsack exactly, so the optimum is often the trivial bound c (or $c-1$ in class 7, where every sum of weights is even and c is odd), as in all 16 instances the literature left open, and a proof must show that no leader choice within the budget prevents a fill that close.

What was known. Exact algorithms for KIP go back to Caprara et al.,[17] Fischetti et al.[18,20] and Della Croce and Scatamacchia.[35] BOBILib lists 392 KIP instances as not optimal, and at 335 of its KIP instances its value is above the optimum, by a median of 8% and up to 672%. Weninger and Fukasawa's combinatorial branch and bound,[26,37] the state of the art, solves 376 of the 392; 14 of these optima were first published by Caprara et al., 26 by Fischetti et al. and 336 in Weninger and Fukasawa's answer files (Section 2.9). The 16 it leaves open in runs with 16 threads and a one-hour limit[19,25] are all in classes 6 and 7, with 200 to 500 items. Weninger and Fukasawa's runs of Della Croce and Scatamacchia's algorithm leave them open too, and the best published lower bounds, in Weninger and Fukasawa's run records, lie 2 to 8 units below BOBILib's values;[19] two-hour runs of Lozano, Bergman and Cire's decision diagrams, in those authors' repository,[38] give weaker bounds. Section 3.2 solves them.

2.3 Multidimensional Knapsack Interdiction

The story. The smuggler's container now has a weight limit and a volume limit, and each good uses some of both; the inspector's budget may also have several parts (hours of different teams). Blocking the most valuable goods is no longer the obvious move: what matters is which goods let the smuggler use both limits well (Figure 5).

The model. Model (2) with several rows in G (the leader's budgets) or in W (the follower's knapsacks), or both. In BOBILib's MKIP instances every row, the leader's and the follower's alike, is a row of one multidimensional knapsack problem (MKP) with K rows: the follower packs items within its rows, and the leader blocks items within its rows, paying for a block what the item would weigh in those rows.

The instances. Both sets come from Fischetti, Ljubić, Monaci and Sinnl;[20] `imkp` is already in their technical report of 2016.[18] Each MKP gives three instances, which differ only in how its K rows are split; the suffix `-0`, `-50` or `-100` names the split. In variant `-0` the leader has the first row and the follower the other $K-1$; in variant `-50` the leader has the first $\lceil K/2 \rceil$ rows and the follower the rest; in variant `-100` the leader has all rows but the last, so the follower has a single knapsack.

- `or` (810 instances), named OR$\langle K\rangle$x$\langle n\rangle$-$\langle\alpha\rangle$_$\langle i\rangle$.dat-$\langle v\rangle$, for example OR10x100-0.25_1.dat-50: built on Chu and Beasley's MKPs from the OR-Library (Operations Research Library),[39,40] with $n \in \{100, 250, 500\}$ items, $K \in \{5, 10, 30\}$ rows, tightness $\alpha \in \{0.25, 0.5, 0.75\}$ and ten instances i for each combination. The weights are integers

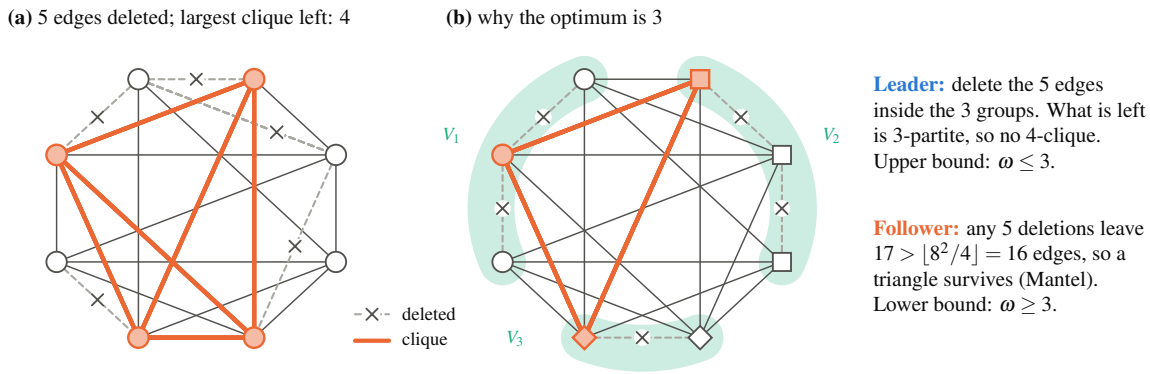


Figure 6. Clique interdiction on an 8-vertex graph with budget 5. (a) A deletion that still leaves a 4-clique. (b) Why the optimum is 3: deleting the 5 edges inside the groups V_1 to V_3 leaves a 3-partite graph, so no 4-clique survives, and any 5 deletions leave more than $\lfloor 8^2/4 \rfloor$ edges, so by Mantel's theorem a triangle survives (Section 3.4, Theorem 5).

between 0 and 1,000; an item's profit is, up to rounding, its average weight over the K rows plus a random amount between 0 and 500. The right-hand side of every row, the leader's budget rows and the follower's capacity rows alike, is α times the sum of its coefficients, rounded to an integer (below, "budget 25%" means $\alpha = 0.25$). So variant `-100` has one follower knapsack, and variants `-0` and `-50` have several.

- `imkp` (144 instances), named after the MKP with the same suffixes, for example weish26.dat-50: built on the SAC-94 suite[41] (the instances hp, pb, pet, sento, weing and weish), with 10 to 105 items and 2 to 30 rows. When $K = 2$ the three splits coincide, and only variant `-50` is kept.

What makes it hard. Everything that makes KIP hard, and more. The follower's problem is now a multidimensional knapsack: a dynamic program would need a state for every combination of used capacities, and the linear programming (LP) relaxation, on which most bounds rest, can overestimate the follower's value by up to the profit of one item per follower row (Theorem 4). The largest OR instances have 500 items and 30 rows.

What was known. Fischetti et al. solved all 144 imkp instances with interdiction cuts in their report of 2016 (10 of them with a longer time limit),[18] but BOBILib lists 52 of them as only feasible, with values up to 23% above the optimum. No OR instance is optimal in BOBILib. Lozano, Bergman and Cire[21] proved 87 of them optimal, one more is closed by their published point and a branch-and-cut bound in their run data (Section 4.4), and they published lower bounds and points for most of the others;[38] Lefebvre and Schmidt[42] published further points; 139 OR instances had no published point.

2.4 Edge Clique Interdiction

The story. In a communication or social network, a clique, a group in which everyone is linked to everyone else, is a tightly knit cluster; large ones shorten the paths along which information travels.[30] The leader can cut up to k links and wants the largest cluster that survives to be as small as possible; the follower, after the cuts, finds the largest cluster left.

The model. The graph $G = (V, E)$ has $n = |V|$ vertices and $m = |E|$ edges. The leader deletes a set D of at most k edges, and the follower's value is the clique number $\omega(G - D)$, the size of a largest clique of the remaining graph. BOBILib's `clique` instances write the follower's problem with a variable y_v^V per vertex and y_e^E per edge, and the leader's with x_e per edge:[33,43]

$$\min_{x \in \{0,1\}^E} \left\{ \max_{y} \sum_{v \in V} y_v^V : y_u^V + y_v^V \le 1 \ (\{u,v\} \notin E),\ y_u^V + y_v^V - y_{uv}^E \le 1,\ x_{uv} + y_{uv}^E \le 1 \ (\{u,v\} \in E) \right\} \quad \text{s.t.} \sum_{e \in E} x_e \le k,$$

with binary y. An edge the leader deletes cannot be used ($y_e^E = 0$), and two vertices that are not joined by a usable edge cannot both be in the clique. So a `clique` instance has m leader variables, $n + m$ follower variables, one leader row and $n(n-1)/2 + m$ follower rows. Figure 6 shows a small instance and why its optimum is 3.

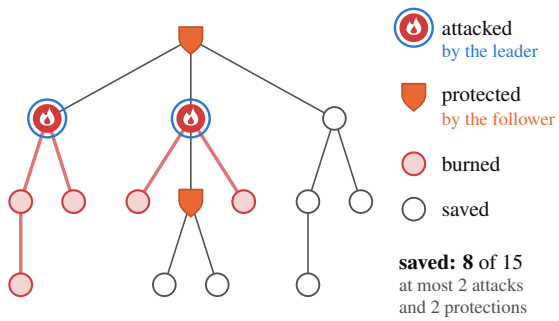


Figure 7. The attack-protect game on a toy tree: the leader attacks, the follower then protects, and the attack spreads through unprotected neighbors. Against the optimal attack shown, the follower's best protection saves 8 of the 15 nodes.

The instances. BOBILib has three sets of edge clique interdiction instances.

- `clique` (60 instances; Fischetti, Monaci and Sinnl[33]), named normalClique$\langle n\rangle$-$\langle 100d\rangle$-$\langle i\rangle$, for example normalClique60-70-2: random graphs with $n \in \{40,50,60\}$ vertices and edge density $d \in \{0.7,0.9\}$, that is $m = \lfloor d\,n(n-1)/2 \rfloor$ edges chosen at random, budget $k = \lfloor m/4 \rfloor$, and ten graphs i for each n and d.
- `bcpins` and `plusbcpins` (Tang, Richard and Smith;[43] Fischetti et al.[33]), named BCPIns_$\langle n\rangle$_$\langle 10d\rangle$_$\langle i\rangle$.txt_trad.txt_K$\langle k\rangle$ and plusBCPIns_$\langle n\rangle$_$\langle 10d\rangle$_$\langle i\rangle$: smaller random graphs, $n \in \{8,10,12,15\}$, the same densities, budget $k = \lceil m/4 \rceil$ and ten graphs for each n and d. The `bcpins` files use a compact follower model with vertex variables only, one row per pair of vertices; the `plusbcpins` files use the model above, strengthened by extra valid inequalities.[33] BOBILib lists all of them as optimal.

What makes it hard. Finding a maximum clique is NP-hard, and the follower's formulation is weak: setting every $y^V_v = 1/2$ and every $y^E_e = 0$ satisfies all its rows, so its LP relaxation is worth at least $n/2$, 20 to 30 here, while every value BOBILib lists is at most 13. The leader picks k of m edges, for example 309 of 1,239, and a general solver must close the gap between the weak relaxation and the true value by branching over these choices. As for knapsack interdiction, the HPR is worth 0.

What was known. BOBILib lists all 60 `clique` instances as feasible, with values 5 to 13; Fischetti et al. used them to test heuristics. Furini et al. gave exact algorithms for vertex and edge clique interdiction,[30,44] the latter strengthened by a Turán-type count of the deletions that leave no clique of a given size in a clique (their Proposition 2, after Hassan et al.[45]), and Zhou et al.'s RLCM is a recent exact method;[31] deciding whether at most k deletions bring the clique number down to a given value is the edge blocker clique problem.[46] None of them ran these instances (the random graphs of Furini et al. and of Zhou et al. have budgets of at most 20% of the edges, BOBILib's 25%), and no optimum of these 60 instances had been published. Section 3.4 proves 57 of them optimal.

2.5 Network Interdiction: The Attack-Protect Game

The story. A fire, a computer virus or a rumor starts at some nodes of a network and spreads along its links. An attacker, the leader, chooses up to Φ nodes to set on fire. A defender, the follower, then protects up to Λ nodes. The fire spreads from the attacked nodes to every neighbor that is not protected, and from there on; a node is saved if the fire never reaches it. The defender maximizes the number of saved nodes and the attacker minimizes it (Figure 7). (Here Φ is a budget, as in BOBILib's row names, not the value function of (2), and the vaccination budget Ω below is not the online game value of Section 3.2.)

The model. The instances come from the multilevel critical node problem of Baggio, Carvalho, Lodi and Tramontani,[47] a three-level game in which a defender first vaccinates Ω nodes, the attacker then attacks and the defender then protects. BOBILib keeps the last two levels, the attack-protect game, on the graph without the vaccinated nodes, as used by Fischetti et al.[15] With $x_v = 1$ for an attacked node, $z_v = 1$ for a protected node and $s_v = 1$ for a saved node (all binary),

$$\min_{x}\Big\{\max_{z,s}\textstyle\sum_v s_v : \sum_v z_v \le \Lambda,\ s_v + x_v \le 1\ (v \in V),\ s_u \le s_v + z_u\ (\text{every edge } \{u,v\}, \text{ both directions})\Big\} \quad \text{s.t. } \textstyle\sum_v x_v \le \Phi.$$

An attacked node is not saved, and a node that is not protected is saved only if all its neighbors are. So an instance on n' nodes and e' edges has n' leader variables, $2n'$ follower variables, one leader row and $n'+2e'+1$ follower rows. (In the files the attack variables are named `y`, the protections `x` and the saved nodes `a`.)

The instances. The set `inter-fire` has 72 instances. Their names, such as tree-50_1-5-5_006, have the form $\langle G\rangle$-$\langle n\rangle$_$\langle\Omega\rangle$-$\langle\Phi\rangle$-$\langle\Lambda\rangle$_$\langle i\rangle$: graph type G (`tree`, a random tree grown one node at a time, or `rndgraph`, a random graph of a given density), $n \in \{25, 50, 80\}$ nodes before vaccination, the three budgets, and the instance number. Each file has $n' = n - \Omega$ nodes, and its rows `Phi` and `Lambda` have the budgets of the name (checked for all 72 files).

What makes it hard. The follower's best protection depends on the whole shape of the attack, and the leader has many attacks to choose from: 2,138,410 sets of at most 5 of 49 nodes in the two unsolved instances. Again the HPR, in which the follower may save nothing, is worth 0.

What was known. BOBILib lists 70 of the 72 instances as optimal. The other 2, two forests with 49 nodes (a tree of 50 nodes without its one vaccinated node) and $\Phi = \Lambda = 5$, had been solved as well: Fischetti et al.'s MIX++ proved all 72 instances of the set optimal, these two at value 26 on four threads within an hour (Table 9 of the online appendix of their paper[15,48]). Section 3.5 proves the two values again by complete enumeration in exact arithmetic.

2.6 Generalized Interdiction

The story. The leader now has its own plans and constraints. Think of a regulator that chooses projects to fund; funding some of them rules out a matching activity of a firm. The firm, the follower, then chooses its activities to maximize its own profit, and the regulator's cost and constraints depend on what the firm does. Unlike in the families above, the two objectives are not opposite.

The model. Fischetti, Monaci and Sinnl's generalized interdiction problem[33] (Figure 8) is, in the notation of (1),

$$\min_{x,y}\ c^\top x + d^\top y \quad \text{s.t.} \quad Ax + By \geq a, \quad y \in \arg\min_{y'} \left\{ f^\top y' : Dy' \geq b,\ x_j + y'_j \leq 1\ (j \in N),\ y' \geq 0,\ \text{integrality} \right\},$$

with binary leader variables and a follower objective f unrelated to d. The set N pairs some leader variables with follower variables: $x_j = 1$ forbids y_j. The leader's rows contain the follower's variables (coupling rows, $B \neq 0$), so the follower's answer can violate them; the follower's rows contain leader variables only in the interdiction rows.

The instances. The set `generalized` has 90 random instances,[33] named general$\langle n_x\rangle$-$\langle n_y\rangle$-$\langle|N|\rangle$-20-20-$\langle i\rangle$, for example general20-30-20-20-20-2: $n_x \in \{20, 30\}$ binary leader variables, $n_y \in \{20, 30\}$ binary follower variables, $|N| \in \{10, 20, 30\}$ interdiction pairs ($|N| \leq \min\{n_x, n_y\}$), 20 leader rows and 20 follower rows, and ten instances for each allowed combination. The coefficients are random integers between -50 and 50. Each leader row has a binary switch variable that relaxes the row by 100,000 and costs the leader 100,000, and each follower row a continuous, unbounded slack variable that costs the follower 100,000 per unit, so every row can always be satisfied, at a price. With the slacks and the interdiction rows, an instance has $n_x + 20$ leader variables, $n_y + 20$ follower variables, 20 leader rows and $20 + |N|$ follower rows.

What makes it hard. The game is not zero-sum, so methods built for interdiction do not apply, and because of the coupling rows the follower's optimal answer can push the leader into paying for slack. The penalties also make objective values large next to coefficients of at most 50; in general20-30-20-20-20-2 the follower's optimum at BOBILib's leader choice is 106,899,953, and at that scale a relative tolerance hides absolute errors of many units (Section 3.1).

What was known. BOBILib lists 12 of the 90 instances as feasible and the rest as optimal; Lozano, Bergman and Cire[21] proved 4 of the 12 optimal.

2.7 Assignment Interdiction

The story and the model. A dispatcher, the follower, assigns 5 workers to 5 jobs, one job each, at the least total cost. Beforehand the leader, within a budget, forbids some worker-job pairs, to make the dispatcher's cheapest assignment as expensive as possible (Figure 9 shows a smaller example). The follower has a binary variable per pair, 20 inequality rows, mostly pairs that form equations, which give every worker exactly one job and every job exactly one worker, and 25 interdiction rows; the leader has one budget row.

The instances. The set `inter-assig` has 24 instances, 2AP05-1 to 2AP05-25 without 2AP05-12, which BOBILib left out as invalid. DeNegre built them from bicriteria assignment problems:[36] the first objective is the follower's cost, the

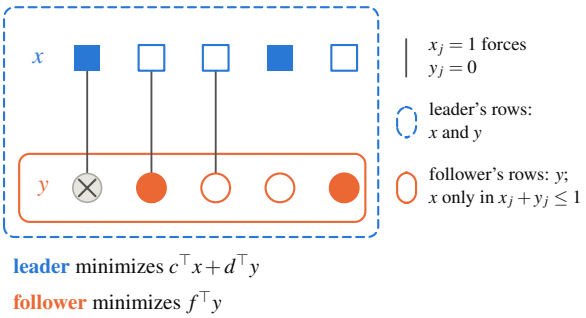


Figure 8. Generalized interdiction: the leader's variables x and the follower's y; for the interdiction pairs, $x_j = 1$ forces $y_j = 0$. The leader's rows contain x and y, the follower's y and, in the interdiction rows, x; the two objectives differ.

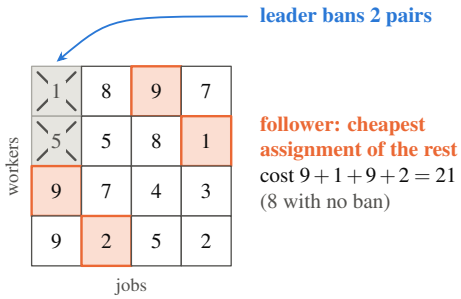


Figure 9. Assignment interdiction on a toy instance with 4 workers and 4 jobs (row = worker, column = job, number = cost). The leader bans two pairs; the follower's cheapest assignment of the rest costs 21 instead of 8.

second gives the leader's costs in the budget row. In 19 of the files the leader's objective is exactly the negated follower cost, a zero-sum game; in the other five it differs on some of the pairs. BOBILib lists all of them as optimal, and this work does not consider them further.

2.8 General Instances from MIPLIB and Other MILPs

The story. Take a real planning model, say a schedule, a network design or a production plan, stored as a MILP in MIPLIB, the library of benchmark MILPs. Hand half of its decisions to a second party, who decides after the first and works against it. The first party, the leader, now has to plan for the worst reply of the second.

The construction. A MILP $\min\{c^\top z : \text{rows, integrality}\}$ with n variables and m rows becomes a bilevel instance by splitting its variables and rows between the two levels and giving the follower an objective (Figure 10). BOBILib has three constructions; the following was checked on every one of its 389 MIPLIB-derived instances.

- `miplib2010` and `miplib2017` (Kleinert and Schmidt;[49] from MIPLIB 2010 and MIPLIB 2017[9,10]), named after the MILP with a suffix: the leader gets the first $\lceil n/2 \rceil$ variables in the order of the file, and the follower the rest. In $\langle X \rangle$_0_100 all rows are the follower's (0% at the leader, 100% at the follower); in $\langle X \rangle$_50_50 the first $\lceil m/2 \rceil$ rows are the leader's and the rest the follower's. In all but two of the _50_50 instances some leader row contains follower variables, so they have coupling rows; the _0_100 instances have none. The third variant, with every row at the leader, leaves the follower without rows and so without linking variables; BOBILib excludes such instances, as it excludes MILPs that are infeasible or have range rows or a zero follower objective, and MIPLIB 2017 models already in MIPLIB 2010.[11]
- `miplib3` (Fischetti et al.;[27,50] from MIPLIB 3,[8] 20 binary models, three instances each), named $\langle X \rangle$-$\langle k \rangle$ with $k \in \{0.100000, 0.500000, 0.900000\}$, for example air03-0.100000: the follower gets the first $\lfloor kn \rfloor$ variables and the leader the rest; all rows are the follower's, so the leader has no rows at all.

In all three the leader keeps the MILP's objective c on all variables, and the follower minimizes $-c$ on its own variables (in the AUX files of 90 instances these coefficients are rounded to six significant digits, for example 83.2776 for $-c_j = 83.2776042$ in gen-ip002_0_100). So for a fixed leader choice the follower makes the leader's cost on its variables as high as it can, and the HPR is the original MILP. BOBILib calls an instance *mixed integer* if it has at least one continuous and one integer variable, and *pure integer* otherwise.

The other general sets. The other general instances are random or small: `denegre` (110 random pure integer instances made with DeNegre's generator,[36] 50 of them from his thesis), `zhang` (30 pure integer instances of Özaltın and Zhang[51]), `misc` (6 small examples from the literature, among them two of Moore and Bard[32]), `xuwang` (100 mixed integer instances of Xu and Wang,[28] with 10 to 460 variables per level) and `xularge` (60 larger ones, 500 to 1,000 variables per level, built the same way by Fischetti et al.[15]). BOBILib lists all of them as optimal except one in `misc`, milp_10_20_50_2310, which is feasible.

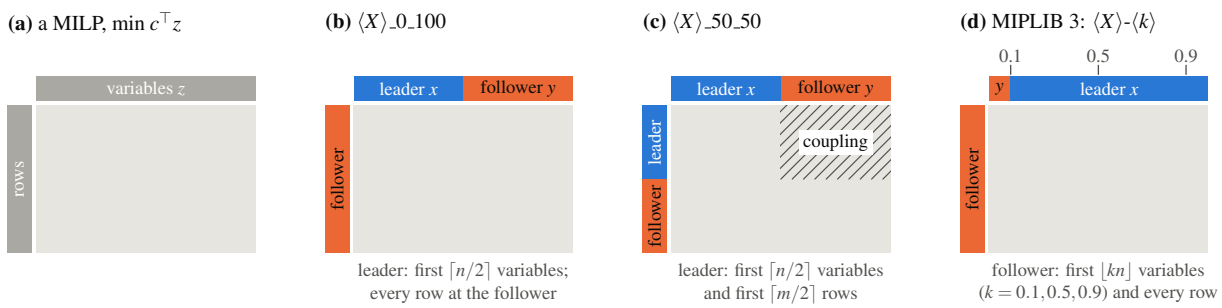


Figure 10. How BOBILib turns a MILP (a) into a bilevel instance. (b) $\langle X \rangle$_0_100: the leader gets the first half of the variables, the follower the rest and every row. (c) $\langle X \rangle$_50_50: the leader also gets the first half of the rows, which then contain follower variables (coupling rows). (d) MIPLIB 3 instances $\langle X \rangle$-$\langle k \rangle$: the follower gets the first fraction k of the variables and every row. The leader keeps the MILP's objective; the follower minimizes its negation on its own variables.

What makes them hard. The HPR alone is a MIPLIB model, and some of these are hard MILPs in their own right; the largest bilevel instances have hundreds of thousands of variables and rows (Table 1). Checking a single point requires solving the follower's MILP to optimality. Linking variables can be continuous, which BOBILib's two solvers do not handle (Section 3.6); all 137 instances with continuous linking variables are MIPLIB-derived and unsolved in BOBILib. In _50_50 instances the coupling rows can make the follower's best answer infeasible for the leader; and in some instances the follower's problem is infeasible or unbounded for every leader choice, so no bilevel feasible point exists at all. This can happen because the follower maximizes the MILP's objective on its own variables, a direction that the MILP's rows need not bound, and in _50_50 instances some of the rows that do bound it have gone to the leader (Section 3.6).

What was known. Almost all of the 266 unsolved general instances are MIPLIB-derived. Their statuses come from BOBILib's one-hour solver runs, and published work had solved 9 of them (Section 2.9).

2.9 What Published Work Had Solved

Table 3 lists the published optima of instances that BOBILib lists as unsolved (Supp. Table S3 names them). The KIP and `imkp` optima come from methods built for knapsack interdiction; the others from general methods for MIBLPs, applied to these instances. For general MIBLPs the methods are branch and bound[32] and branch and cut,[13,52] intersection cuts,[15,27,50] value-function reformulations,[53,54] decision diagrams,[21,55] a Dantzig–Wolfe single-level reformulation[42] and heuristics such as a penalty alternating direction method (PADM).[49] Published points without a proof of optimality are used in two ways: to decide whether a point of this work is the first one known (Section 3.1), and as the published upper bounds of the OR comparison (Section 4.4). Lozano, Bergman and Cire's repository lists 344 instances as solved to optimality.[38] Of these, the KIP and `imkp` instances had been solved before[17–20] and p0201-0.100000 by Fischetti et al.;[15] Table 3 credits Lozano, Bergman and Cire with the others, with the OR optima that appear only in their table of solved instances and with one closure, 96 in all.

The 376 KIP optima come from three first publications. Caprara, Carvalho, Lodi and Woeginger published 14 of the `cclw` optima.[17] Fischetti, Ljubić, Monaci and Sinnl's technical report of 2016, the preprint of their 2019 paper, published the 25 `inter-kp` optima and interdiction55-3, whose value Caprara et al. list without a proof.[18,20] The same report introduced the `imkp` set and is also the first publication of the 52 `imkp` optima that BOBILib lists as feasible (Tables 4–6; 10 of them are given in two captions, from runs with a longer time limit; Supp. Sec. C.6). The 336 `kp` optima appear first in the answer files of Weninger and Fukasawa's code, whose repository was public by 7 October 2022;[19] the preprint they posted that day gives counts per class only.[56] Della Croce and Scatamacchia had reported 266 of these `kp` instances solved, as counts per group without values (for 6 of them this assumes that a misprinted "6/18" means 6 of 6; Supp. Sec. B.9).[35] Three documents that came before the answer files could not be read in full: the journal version of Fischetti et al.'s report with its online supplement, Della Croce and Scatamacchia's IPCO 2019 paper, and the version of record of the paper that introduced the `kp` set.[20,33,57] That they list no `kp` values is inferred from their abstracts and from the accepted manuscript of the last, which says the optimal values are not known (Supp. Sec. B.9).

3. METHODS

3.1 Reconciling the Statuses and Verifying Every Claim

Published results. For every family I compared BOBILib's status and value with the per-instance data the authors publish: answer files and run records,[19] tables, solution files[38] and run files.[42] An instance counts as solved in published work when the data its authors released, in the paper or in the files that go with it, prove its optimal value: a claimed optimum, or a published point together with a bound from their runs that meets it. Lefebvre and Schmidt's run files need more care (below). Two of Lozano, Bergman and Cire's optima need care. For OR10x100-0.75_4.dat-0 the lower bound in their paper's table exceeds their point, and a branch-and-cut bound in their run data closes the instance (Section 4.4). For p0548-0.900000 their decision diagrams report 48,851 at a point whose follower answer is not optimal and passes only BOBILib's tolerance, a published class (iii) point (below); their branch-and-cut run proves 48,853, the value used here. Each KIP instance was matched to Weninger and Fukasawa's file by its capacities and the multiset of item triples (profit, follower weight, leader cost); every match is unique. The other KIP sources use the same instance names; BOBILib's files were checked against the bkpsolver files and, for `cclw`, against Carvalho's instance file (Supp. Sec. B.9). For 449 of the 531 published optima a verified solution file at the published optimum is in the data package (for all 376 KIP instances it was rebuilt from Weninger and Fukasawa's leader choices). For the other 82 (52 imkp, 18 OR, 8 MIPLIB-derived and 4 generalized interdiction instances) only the published value is used. The literature's optimality proofs are not repeated here.

Lefebvre and Schmidt's run files. Their per-instance run files cover most of BOBILib, but not every row can be read at face value: rows that end at the time limit can carry the status "optimal" with a value that no bilevel feasible point has (the value 0 on 366 OR instances), and rows of runs that ended in time, on about a hundred other instances, almost all of them outside the boundedness assumption of their method, contradict verified points or certificates (Supp. Sec. A.6). Their own analysis discards time-limit rows except in one best-of-all-methods comparison (their Figure 6), and this paper discards them throughout: their values count as published points only from runs that ended before the limit. Their files hold no solutions, so an optimum that a run reports before the limit, with its value equal to its bound, counts as published only where a point with that value is verified here. This holds for ex10_0_100, optimal at 100 in their run file `results-MILP.csv` although their paper does not name it; it is credited to them, and the MIPLIB bound confirms it independently (Section 4.6). Their runs also report neos-950242_0_100 optimal, which an earlier BOBILib release had already listed as optimal and to

Table 3. Published optima of instances that BOBILib lists as feasible or open.

Source	Method	Instances	Optima
Weninger and Fukasawa, answer files[19]	combinatorial branch and bound	KIP, set `kp`‖	336
Fischetti, Ljubić, Monaci and Sinnl, report[18]	interdiction cuts	KIP, sets `inter-kp` and `cclw`*	26
		MKIP, set `imkp`	52
Caprara, Carvalho, Lodi and Woeginger[17]	a sequence of MILPs	KIP, set `cclw`	14
Lozano, Bergman and Cire[21]	decision diagrams	MKIP, set `or`	88†
		generalized interdiction	4
		MIPLIB 3	4
Fischetti et al.[15,48]	branch and cut	MIPLIB 3, network§	5
Lefebvre and Schmidt[42]	Dantzig–Wolfe reformulation	ex10_0_100¶	1
An earlier BOBILib release	solver runs	neos-950242_0_100‡	1
Total			531

Each optimum is credited to its first publication: the earliest public document (paper, appendix, thesis or code repository with result files) that reports it proved optimal and lists its value, among the per-instance sources compiled here (rule in Supp. Sec. H.2; KIP and `imkp` in Supp. Secs. B.9 and C.6). Earlier reports without values and later proofs are named in the data package. The rule covers only instances BOBILib lists as feasible or open; the 207 KIP and 92 `imkp` instances it lists as optimal keep BOBILib as the source, although earlier documents give the same values. ‖The repository was public by 7 October 2022; 266 of them were reported solved earlier, without values (for 6 of them this assumes that a misprinted "6/18" means 6 of 6; Supp. Sec. B.9).[35] *interdiction55-3, whose value Caprara et al. list without a proof. †87 listed as optimal in their published data (most in their collection of optimal solution files, the others only in their table of solved instances), and OR10x100-0.75_4.dat-0, closed by their published point and a branch-and-cut bound in their run data (Section 4.4). §air03-0.100000, p0201-0.100000 and p0282-0.500000, solved with a more aggressive setting of MIX++ (caption of their Table 6; the last two were proved optimal again later by Vásquez, Lozano and van Hoeve,[55] p0201-0.100000 also by Lozano, Bergman and Cire), and tree-50_1-5-5_006 and tree-50_1-5-5_010 (Table 9 of their online appendix; re-proved here, Section 3.5). ¶From their run files (Section 3.1); the MIPLIB bound of Section 3.6 and a verified point confirm it. ‡Its BOBILib point (not re-verified here) also meets the MIPLIB bound of Section 3.6 (Supp. Table S11).

Table 4. Tolerances of the checks.

Check	Tolerance
`bobi verify`: rows, bounds and integrality of the HPR at a file's point	10^{-6} absolute
`bobi verify`: follower optimality, test (3) (differences up to 10^{-9} count as zero)	10^{-4} relative
`bobi verify --tight`: follower value above every optimum the solver's proof allows	$\max(10^{-6}, 10^{-9}\|\varphi\|)$
`bobi certify --tight` (general points): follower value above its proven dual bound	10^{-6} absolute
`bobi certify --tight`: optimistic tie-break, leader value F above its proven bound	$10^{-6} + 10^{-9}\|F\|$
BOBILib's checker: rows and integrality (MIPLIB solution checker)	10^{-4}, scaled by the row
BOBILib's checker: follower optimality, test (3)	10^{-4} relative

which it is credited. Their optima for ex10_50_50, where no point is known, and mine-166-5_50_50, whose point their own check file rejects, are not counted. Their exact runs also end with status "infeasible", before the limit, on 11 of the 40 instances shown here to have no bilevel feasible point (Section 3.6). These instances, too, lie outside the boundedness assumption of their method, the same status is contradicted by verified points on 90 other instances, and their paper does not state these results, so they are not counted as published.

Verification. Every solution file of this work uses BOBILib's format and passes `bobi verify`, a verifier written for this work after BOBILib's own feasibility checker (Supp. Sec. A.2). It is separate from BOBILib's solvers, but not from the rest of this work's code: it shares the instance reader and the exact knapsack oracle with the solvers. It checks (a) every row, bound and integrality requirement of the HPR at the file's point; (b) the stated leader objective; and (c) follower optimality: with the leader's variables fixed, the follower's problem is solved again, with one follower row by an exact knapsack oracle, a branch and bound with Dantzig's LP bound that falls back to dynamic programming,[58,59] and otherwise as a MILP with HiGHS.[60] BOBILib's checker accepts the file's follower value φ_{sol} if it passes a relative test against the proven optimum φ_{check},

$$\frac{|\varphi_{\text{sol}} - \varphi_{\text{check}}|}{10^{-10} + |\varphi_{\text{check}}|} \leq 10^{-4}. \tag{3}$$

When the follower's values are large, this test accepts a follower answer that is far from optimal in absolute terms, and the leader can profit from it. So every file is also held to a *tight test* (`bobi verify --tight`): the follower's value may exceed every follower optimum that the solver's proof allows by at most $\max(10^{-6}, 10^{-9}|\varphi|)$ (Table 4). All 1,478 current solution files of the data package pass it. At first 25 of them did not (17 without and 8 with coupling rows): their follower sat up to $10^{-7}(1+|\varphi|)$ above its optimum, the slack allowed by the best-response step of the methods of Section 3.6, and they passed only BOBILib's test. These files and neos17_50_50, which already passed, 26 in all (17 without and 9 with coupling rows), were re-derived with the follower solved to zero gap, at the same leader values except for pg_50_50 and neos-2978193-inde_50_50. Every leader value got worse, by at most 1.4×10^{-6} relative (1,000 units for core2536-691_0_100, whose values are near 10^{10}; Supp. Secs. E.3 and E.4; the old files are kept as superseded). Now every file's follower lies at most 1.3×10^{-9} above its optimum, except neos17_0_100 and neos-5188808-nattai_0_100 (below 10^{-6}). The full protocol is in Supp. Sec. A.3.

Certificate classes. The results rest on arguments of different rigor, and every claim is put into one of three classes (Supp. Sec. A.4):

(i) An *exact certificate* is checked in exact rational or integer arithmetic, or consists of a mathematical proof together with an exhaustive computation in integer arithmetic. No tolerance enters it; it still relies on code that is tested, and for its main lemmas backed by machine-checked proofs, but not itself verified.

(ii) A *solver-certified numerical* result rests on a floating-point LP, MILP or constraint programming solver with tolerances: a proven dual bound, a verdict, or a point whose rows hold within 10^{-6} and whose follower passes the tight test. Such a point is feasible within these tolerances, not exactly; the result is correct if the solver's numerics are.

(iii) A *benchmark-accepted* point passes BOBILib's tolerance-based checker only because of its tolerances, and need not be bilevel feasible. BOBILib's own solution file of the generalized interdiction instance general20-30-20-20-20-2 is an example: its follower value 106,899,984.14 lies 31 units (2.9×10^{-7} relative) above the follower's optimum 106,899,953 at the same leader choice, and a row with right-hand side -245 is violated by 3.3×10^{-4}; both pass BOBILib's tolerances, which are relative (Supp. Sec. E.5).

Table 5. Certificate class of each kind of result, over the 878 unsolved instances with an outcome (column `certificate_class` of the data package).

Result	Basis	Class	Instances
Knapsack interdiction optima	exhaustive integer search, Lean-proved pruning lemmas, two integer-only provers	(i) exact	16
Edge clique interdiction optima	Mantel's and Brouwer's theorems, integer-checked case trees, partition files	(i) exact	57
No bilevel feasible point	recession direction checked in rational arithmetic	(i) exact	40
Published optima, re-proved exactly (network)	every attack enumerated, exact tree program	(i) exact	2
Optima at the MIPLIB bound	exact model comparison, MIPLIB's floating-point optimum	(ii) solver	6
OR points, several follower knapsacks	follower optimum by HiGHS	(ii) solver	140
General points	follower and tie-break by HiGHS, tight test	(ii) solver	88
Other published optima with a verified file	published bound; point verified here	(ii) solver	447
Published optima, value only	as published	(ii) solver	82
Total: class (i) 115, class (ii) 763, class (iii) 0			878

A result is in the weakest class of the steps it uses. Published optima are used as published: they rest on their authors' software and its numerics, and for 82 of them not even the point is verified, so they are listed in class (ii) as the weakest class that fits. The exception are the 2 network optima, which this work re-proves by complete enumeration (Section 3.5); they are listed in class (i). As in BOBILib and MIPLIB, a verified point that meets a lower bound of class (i) or (ii) is called optimal. Table 5 gives the class of every settled instance and every new point; the text gives that of each bound (Section 4.4 for the OR lower bounds). No result of this work is in class (iii).

3.2 Knapsack Interdiction: A Robust Subset-Sum Search

All 16 KIP instances open in the literature are subset-sum instances: the follower's value is the largest sum of surviving weights that fits its capacity. The search answers a yes/no question: given an integer ℓ, does every leader choice within budget leave the follower a subset sum in the band $[\ell, c]$? A "yes" proves OPT $\geq \ell$; a "no" comes with a leader choice whose follower value is below ℓ. The loop starts from Weninger and Fukasawa's greedy leader choice and asks the question with ℓ equal to the value of the best leader choice found so far; it stops at the first "yes", which proves optimality (Supp. Cor. B.25). For each of the 16 instances it stopped at BOBILib's value, c or $c-1$, so the band $[\ell, c]$ has width 0 or 1. The rules below hold for any band; for an instance whose optimum lies far below c the band is wider, and the loop may need several searches.

Count formulation. Subset-sum instances have many items with the same weight. Such items are interchangeable for the follower, so the leader should delete the cheapest copies first, and the search branches on how many copies of each weight survive rather than on which items are deleted (Figure 11a; Supp. Prop. B.4, which also divides the weights by their greatest common divisor). This removes many symmetric branches. Weninger and Fukasawa's code already merges items equal in profit, weight and cost into counted copies (its option `-dedup`, since 2023), and its current version (August 2026, not described in their paper) breaks symmetry by a dominance rule that, for items of equal weight, contains Lemma 1(ii);[19] here a class holds all items of one weight, whatever their costs.

Lemma 1 (Monotonicity and cheapest copies)**.** *(i) Blocking more items never helps the follower: $X \subseteq X'$ implies $\Phi(X') \leq \Phi(X)$. (ii) For a class Q of identical items and any leader set X there is a leader set X' that agrees with X outside Q, blocks as many copies of Q, blocks only copies no costlier than every copy it leaves, costs no more than X, and has $\Phi(X') = \Phi(X)$.*

Proof in Supp. Lemmas B.1 and B.2; Lean: `follower_antitone`, `cheapest_first`.

Windows. At a node of the search the decided classes have left survivors M whose subset sums are known. A run of consecutive reachable sums only grows when a copy of value v is added, as long as v is at most the run's width, as in Brown's criterion for complete sequences[61] (Figure 11b; Lean: `window_extend`, `window_extend_all`). If the leader cannot afford to delete enough weight to stop such a window before it reaches the band, the node is safe whatever the leader does (Supp. Prop. B.8).

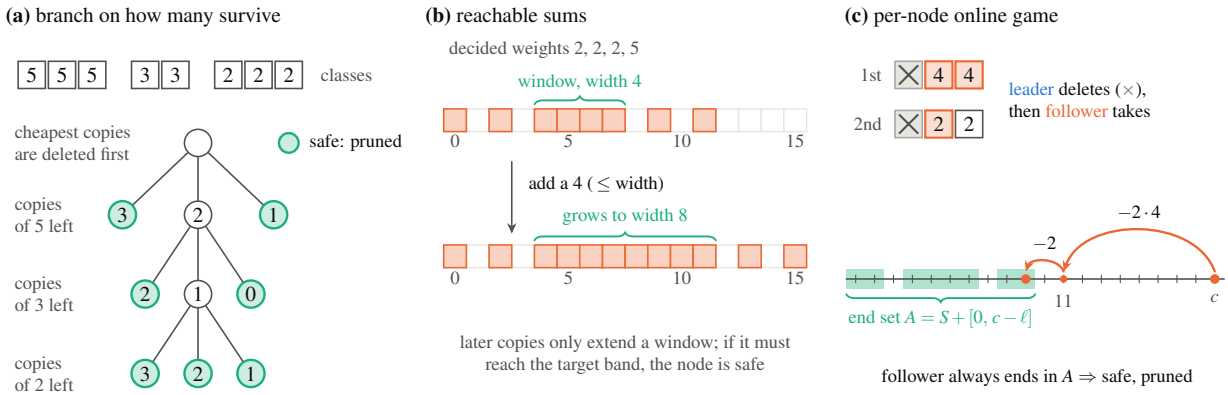


Figure 11. The search that proves the 16 new knapsack interdiction optima, on toy numbers. (a) It branches on how many copies of each weight survive. (b) A window of consecutive reachable sums only grows as copies are added. (c) At each node an online game, in which the leader deletes one weight class at a time, decides whether the follower can always end in the set A; if so the node is pruned.

The online game. The strongest rule is an online relaxation due to Weninger and Fukasawa. Fix an order of the items. In the online game the leader decides item by item whether to block it, and the follower, who has seen every earlier move, decides whether to take each unblocked item. The leader may react to the follower's earlier choices, which it cannot do in the real game, so the follower's value in the online game is a lower bound:

Theorem 2 (Online lower bound[26])**.** *For every list o of distinct items, the value $\Omega(o,\beta,c)$ of the online game, computed by backward induction, satisfies $\Omega(o,\beta,c) \le \Phi(X)$ for every leader set X within the budget β. Hence $\Omega(o,\beta,c) \le \mathrm{OPT}$.*

Proof in Supp. Thm B.9; Lean: `online_le_K_any`, `online_lower_bound`.

Tables with an end set. For subset-sum instances I play the online game class by class, in the count formulation and with a target. Weninger and Fukasawa's code plays it the same way on classes of identical items, the leader choosing how many copies to block and the follower how many to take;[19] new here are classes of one weight with different costs, the end set built from a node's decided sums, and the table rebuilt for that end set at every node. At each class the leader announces how many copies it deletes; the follower then takes some of the surviving copies, which lowers a residual t; the follower wins if the final residual lies in an *end set* A. Let $G_A(Q,\beta')$ be the set of residuals from which the follower wins when the classes Q are played in a given order and the leader has budget β'. Then every deletion within β' leaves a subset sum s with $t - s \in A$ (Supp. Thm B.12; Lean: `GA_sound`, and `GA_sound_perm` for any order of play). A follower who wins against a budget also wins against any smaller one, so the table is stored as one number per residual: the largest budget against which the follower still wins. These thresholds satisfy a min-max recursion over the classes, and one backward pass computes them for all residuals (Supp. Thm B.17; Lean: `thr_iff`, `GA_iff_thr`).

The per-node rule. The decided survivors at a node already reach a set S of subset sums, and the follower wins with any total in $[\ell,c]$. So the node's end set is $A_{\text{node}} = \{a \in [0,c] : s \le a \le s + (c-\ell) \text{ for some } s \in S\}$, and the game starts from the residual c: if it ends at $a \in A_{\text{node}}$, the follower's total $c-a$ plus a suitable $s \in S$ lies in the band.

Theorem 3 (Per-node rule)**.** *At a node with decided sums S and residual budget β', let π be any order of the undecided classes. If β' is at most the threshold of the residual c in the table for π with end set A_{node}, then every completion of the node within β' leaves a subset sum in $[\ell,c]$: the node can be pruned, and it stays safe for every smaller budget.*

Proof in Supp. Thm B.19; Lean: `node_table_sound_perm` and `node_table_threshold_sound_counts`.

Figure 11c shows the rule on toy numbers. The decided sums are $S = \{0,3,5,8\}$ and the band is $[18,19]$, so $A_{\text{node}} = \{0,1,3,4,5,6,8,9\}$. Three copies of 4 and three of 2 are undecided, each copy costs 1 to delete, and the leader has budget 2. The game starts at the residual 19. In the play shown, the leader deletes a 4 and the follower takes the other two (residual 11); the leader deletes a 2 and the follower takes one of the two left (residual $9 \in A_{\text{node}}$). The follower's total $8+2=10$ plus $s=8$ is 18, in the band. Checking every play shows that the follower can always end in A_{node} at budget 2, so the node is pruned.

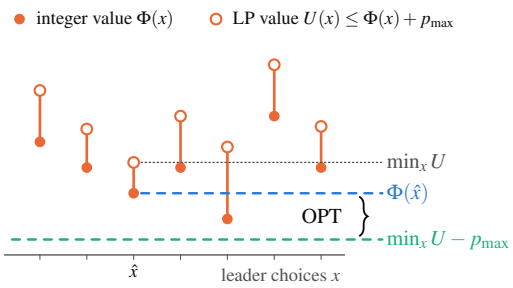


Figure 12. The LP bracket for one follower knapsack (made-up numbers): the follower's LP value $U(x)$ exceeds its integer value $\Phi(x)$ by at most $p_{\max}$, so the optimum lies between $\min_x U - p_{\max}$ and $\Phi(\hat{x})$, where $\hat{x}$ minimizes U.

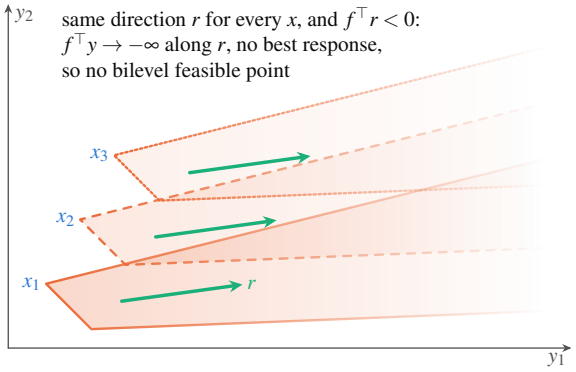


Figure 13. Why some MIPLIB-derived instances have no bilevel feasible point (Section 3.6): the follower's region moves with the leader's choice x but keeps the same direction r, and $f^\top r < 0$ leaves the follower without a best response for every x.

The table is rebuilt from the node's own sums at every node that would otherwise branch, and its threshold is cached, so later nodes with the same depth and sums and a budget below the threshold are pruned at once. The search also uses cheaper rules: the *direct* rule (the decided sums S already meet $[\ell, c]$), guaranteed copies, the window rule and the cache (Supp. Thm B.23). The order of play does not affect soundness but matters a great deal for strength. The search plays the classes that are cheapest to delete per unit of weight first: the follower then sees the leader's likely deletions before it commits, and keeps the expensive classes for fine adjustment. On interKP-200-100-6-2, tables built only at the root needed 1,128,566 nodes when played largest weight first and 513,834 when played cheapest first; per-node tables need 303. A node is declared "blocking" only at a leaf no rule prunes, which yields a leader choice below ℓ; so a search that ends within its node limit is exact (Supp. Thm B.24). The resulting optima are computational proofs supported by formally verified lemmas, not formally verified solutions: the search is exhaustive and works in integer arithmetic except for the window rule, its main pruning lemmas are proved in Lean, and two integer-only provers that do not use the window rule repeat it (Section 3.7). They are exact certificates (class (i)).

3.3 Multidimensional Knapsack Interdiction: A Linear Programming Bracket

Let $U(x)$ be the value of the follower's LP relaxation in (2), in which the follower may take fractions of items, and $p_{\max}$ the largest profit. Dualizing the follower's LP gives

$$U(x) = \min_{\lambda \ge 0} \; c^\top \lambda + \sum_{j=1}^{n} \max\bigl(0,\, p_j - (W^\top \lambda)_j\bigr)(1 - x_j),$$

and for binary x the product linearizes exactly, so $U^* = \min\{U(x) : Gx \le g\}$ is the value of a single MILP (Supp. Prop. C.2). Caprara et al.'s algorithm for KIP rests on this dualization,[17,35] and it is the classical single-level reformulation of network interdiction.[22,23] For one follower knapsack, Weninger writes U^* the same way, as one minimization over (x, λ), and proves OPT $\ge U^* - p^*$ with $p^* \le p_{\max}$, the case $m = 1$ of Theorem 4; he also gives the dual form for several follower rows.[25] An optimal vertex of the follower's LP has at most m fractional items when the follower has m rows, which bounds the gap between U and Φ (Figure 12).

Theorem 4 (LP bracket; for $m = 1$ Weninger[25]). *With m follower rows, $U(x) - m\,p_{\max} \le \Phi(x) \le U(x)$ for every leader choice x. Hence for any lower bound $L \le U^*$, for example a MILP solver's proven dual bound, and any leader-feasible $\hat{x}$: $\lceil L - m\,p_{\max} \rceil \le \text{OPT} \le \Phi(\hat{x}) \le U(\hat{x})$.*

Proof in Supp. Lemma C.3 and Thm C.4; not formalized.

With one follower knapsack (variant `-100`, $m = 1$) the bracket is at most one item's profit wide when the MILP is solved to optimality ($L = U^*$ at its minimizer $\hat{x}$); with a time limit its width is $\Phi(\hat{x}) - \lceil L - p_{\max} \rceil$. In floating point, L is HiGHS's proven dual bound, used only when it is finite and lowered by a safety margin of $10^{-6}\max(1, |L|)$ before rounding up, so

Table 6. Edge counts behind the clique lower bounds. Every instance of a group has the same n, m and k. Mantel's theorem needs $m-k > \lfloor n^2/4 \rfloor$; Brouwer's needs $m-k \geq t_3(n) - \lfloor n/3 \rfloor + 2$.

n	Density	m	k	$m-k$	$\lfloor n^2/4 \rfloor$	$t_3(n)$	$t_3(n) - \lfloor n/3 \rfloor + 2$	Optimum
40	0.7	546	136	410	400	533	522	3
50	0.7	857	214	643	625	833	819	3
60	0.7	1,239	309	930	900	1,200	1,182	3*
40	0.9	702	175	527	400	533	522	4
50	0.9	1,102	275	827	625	833	819	4
60	0.9	1,593	398	1,195	900	1,200	1,182	4

* Except three instances of this group, which are open, with value 3 or 4 (Section 4.5).

that its rounding error cannot push the bound past an integer (Supp. Sec. C.2); the bound is solver-certified (class (ii)). A second lower bound extends Theorem 2 to several leader rows: merge them into one surrogate row[62] with nonnegative multipliers, then round the leader's costs down and its budget up, and the follower's weights up and its capacity down. Each step only helps the leader, so the online game on the merged data is still a lower bound. The rounding goes in the safe direction and the game is solved in integer arithmetic (Supp. Thm C.5; the online bound is formalized, the merging and rounding are not), so this bound is an exact certificate (class (i)). Upper bounds come from a first-improvement local search on the leader's choice that evaluates each candidate with the exact knapsack oracle, so every point it reports is certified.

With several follower knapsacks (variants `-0` and `-50`) the bracket's width $m\,p_{\max}$ is too large for a useful lower bound, and only the literature's lower bounds are used. The minimizer $\hat{x}$ of the MILP is still a good leader choice: whenever HiGHS solves the follower's integer problem at $\hat{x}$ to proven optimality, the point is certified (class (ii); Supp. Sec. C.4).

3.4 Clique Interdiction: Extremal Graph Theory and Partitions

After any deletion at least $m-k$ edges remain, and extremal graph theory says how many edges force a clique of a given size.

Theorem 5 (Clique bounds)**.** *Let G have n vertices and m edges, and let the leader delete at most k edges.*

(i) (Mantel[63]) If $m-k > \lfloor n^2/4 \rfloor$, every remainder contains a triangle, so the value is at least 3.

(ii) (Partitions) If the vertices can be split into r parts with at most k edges inside the parts, deleting those edges leaves an r-partite graph, so the value is at most r.

(iii) (Brouwer[64,65]) If $n \geq 7$, $m-k \geq t_3(n) - \lfloor n/3 \rfloor + 2$, and every split of the vertices into 3 parts has more than k edges inside the parts, every remainder contains a K_4, so the value is at least 4.

Proof in Supp. Thms D.2 and D.6 and Prop. D.3. Parts (i) and (ii) are proved in Lean (`mantel`, `triangle_survives` and `cliqueFree_of_partition`); part (iii) is not formalized, as it rests on Brouwer's theorem.

Here $t_3(n)$ is the number of edges of the balanced complete 3-partite graph on n vertices, the Turán number.[66] Brouwer's Theorem 2 of 1981, in an unrefereed report,[64] says that a K_4-free graph on $n \geq 7$ vertices with at least $t_3(n) - \lfloor n/3 \rfloor + 2$ edges is 3-partite. Tyomkyn and Uzzell give a new proof of this statement in a refereed journal (their Theorem 1.1, for K_{r+1}-free graphs; here $r = 3$),[65] and the same threshold follows from the results of Hanson and Toft, of Kang and Pikhurko, and of Amin, Faudree, Gould and Sidorowicz.[67–69] So in (iii) a K_4-free remainder would be 3-partite, the deletion would contain every edge inside the parts of some 3-partition, and that takes more than k deletions. Table 6 gives the numbers. At density 0.7 Mantel's theorem applies. At density 0.9 Brouwer's threshold is met, while $m-k$ stays below $t_3(n)$, so Turán's theorem alone would not give a K_4. Figure 6 (Section 2.4) shows parts (i) and (ii) on a small graph.

The last condition of (iii) reduces to finitely many cases. Counting the non-edges inside the parts, a 3-partition with given part sizes has few edges inside its parts exactly when few edges of the sparse complement graph join different parts. Only a handful of part-size vectors leave any room, and for each of them a partition with at most k inside edges would need at most a few complement edges between the parts (Supp. Prop. D.7). That gives 170 cases (instance, part sizes), each of which must have no solution. Each case has a certificate: an explicit branch-and-bound tree whose leaves are infeasible by a simple counting bound. A checker of 174 lines of code that shares no code with the prover or the rest of this work reads

the instance files itself, re-derives the cases and checks every tree in integer arithmetic: all 170 pass (134,368 tree nodes in all, the largest with 9,888; 3.1 s). Given Brouwer's published theorem, the lower bound 4 is therefore an exact certificate (class (i)). The same cases were also decided infeasible by CP-SAT[70] and SCIP[71,72] on two different models, by an exact depth-first search and, in a separate review session (Section 3.7), by HiGHS and a branch and bound in C. For the upper bounds, a tabu search looks for 3- and 4-partitions with few inside edges. Each solution file deletes exactly the inside edges of a partition and states a maximum clique of the remainder, computed exactly and checked again with two exact clique searches.[73–75]

3.5 Network Interdiction: Every Attack, with an Exact Tree Program

For a fixed attack, the follower's best response in the attack-protect game is a closure problem: the burned nodes are the smallest set that contains the attacked nodes and is closed under spreading to unprotected neighbors (Supp. Lemma F.1). On a forest it is solved exactly by a dynamic program (DP) over the states (protected, saved) of each node, with the number of protected nodes as a knapsack dimension (Supp. Sec. F.3). The two instances BOBILib lists as feasible, which Fischetti et al.'s MIX++ had already solved (Section 2.5), are forests with 49 nodes and $\Phi = \Lambda = 5$, so all 2,138,410 attacks of at most Φ nodes can be enumerated (Figure 7, Section 2.5, shows the game). Every follower point is checked against the model as read from the files, and at the minimizing attack the follower's MILP is solved again by HiGHS. The minimum over all attacks is then the optimum (Supp. Thm F.4), an exact certificate (class (i)) of the published value. The data package counts the two instances as published optima, in class (i) because of this re-proof (Section 3.1).

3.6 General Instances: Recession Directions, the MIPLIB Bound and Feasible Points

No bilevel feasible point. The leader's variables enter the follower's rows only as constants. So a direction along which the follower can improve without limit works for every leader choice at once (Figure 13).

Theorem 6 (Recession direction[27,28])**.** *Let r be a rational direction of the follower's variables that keeps every follower row and bound satisfied when added to a feasible point ($D_i r \geq 0$ for a row with a finite lower side, $D_i r \leq 0$ for a row with a finite upper side, and the same sign conditions for the variable bounds), and let $f^\top r < 0$. Then for every leader choice the follower's problem is infeasible or unbounded, so the bilevel problem has no feasible point.*

This is Theorem 1 of Fischetti et al.,[27] who test it with one LP and extend Lemma 2 of Xu and Wang;[28] it is written here for rows with a finite lower side, a finite upper side or both. Proof in Supp. Thm E.1, after Meyer;[29] not formalized. New here are its systematic application to BOBILib's unsolved general instances and the exact rational check of each direction.

The integrality of the follower is no obstacle because r is rational: a multiple of r keeps the integer variables integral. The directions are found by an LP solver in floating point. A script that reads the instance files independently of the solver code converts each direction to exact rationals and checks the conditions of Theorem 6 in rational arithmetic. For supportcase42_0_100 and supportcase42_50_50 the solver's direction kept small rounding residuals; a search in exact arithmetic found a simpler one, a single continuous follower variable with negative follower cost that appears only in one upper-bounded row, with coefficient -1. All 40 certificates are exact (class (i)).

The MIPLIB bound. Every bilevel feasible point is feasible for the HPR, so HPR $\leq$ OPT, and a bilevel feasible point with value HPR is optimal (Supp. Lemma E.3). When BOBILib builds an instance from a MIPLIB problem and changes nothing else, the HPR *is* that MIPLIB problem, and its published optimum (MIPLIB 2017 solution file, version 37 of 24 September 2026)[10] is a lower bound on the bilevel optimum. This is checked, not assumed: a script compares the bilevel instance with the MIPLIB 2017 model in exact rational arithmetic (variables, rows, coefficients, sides, bounds, integrality and objective). The published optimum itself, however, is a floating-point result of MILP solvers. Optima obtained this way are optimal by the accepted conventions of the two benchmark libraries (class (ii)), not by an exact-arithmetic proof. Where the objective takes integer values only, as for noswot, ex10 and neos-1445765, or multiples of 1,000 only, as for pigeon-10, the published bound rounds up to such a value, and the result is safe if that bound is valid.

Feasible points. Without coupling rows, a bilevel feasible point follows from any HPR point at which the follower's problem has an optimal solution: fix the leader's part, solve the follower to optimality, and choose among the follower's optima the one best for the leader, the optimistic tie-break (Supp. Prop. E.4). Unlike BOBILib's two solvers, this handles continuous linking variables. With coupling rows, all follower optima at the HPR's leader values may violate a coupling row. For fixed values of the linking variables at which the follower's problem has an optimum, two solves, the follower's

Table 7. What is machine-checked. Lean theorem names are in the namespace `Bobi` of the project `bobilib/lean`; Supp.: section of the supplementary document.

Result	Lean theorems	Supp.
Monotonicity, cheapest copies first (Lemma 1)	`follower_antitone`, `cheapest_first`	B.1
Windows of reachable sums	`window_extend(_all)`, `window_survivors`	B.2
Online lower bound (Theorem 2)	`online_lower_bound`, `online_le_K_any`	B.3
Tables with an end set, any order of play	`H_sound(_any)`, `G_sound`, `GA_sound`, `GA_sound_perm`, `covers_sound`, `band_covers_sound`	B.4
Threshold form	`thr_iff`, `GA_iff_thr`, `covers_threshold_sound(_perm)`, `band_covers_threshold_sound(_perm)`	B.5
Per-node rule (Theorem 3)	`node_table_sound`, `node_table_sound_perm`, `node_table_threshold_sound_counts`	B.6
Mantel's theorem, deletion form (Theorem 5(i))	`mantel`, `triangle_survives`	D.2
Partition upper bound (Theorem 5(ii))	`cliqueFree_of_partition`	D.3

Not formalized: the search code itself; the reduction to the count formulation with its scaling by the greatest common divisor (Supp. Prop. B.4); the fractional deletion bound in the window rule; the direct, guaranteed-copies and cache rules and the node-state lemma (Supp. Thm B.23 and Lemma B.22); restricted copies (an option no recorded run uses); the LP bracket and the merged-row bound; Brouwer's theorem and the reduction to part-size cases; the network DP; the recession directions and the MIPLIB comparison.

problem and then one MILP for the optimistic best response with the coupling rows, find the best bilevel feasible point with those values or show that there is none (Supp. Prop. E.5). A portfolio of heuristics, `bobi coupled`, searches for usable linking values: the HPR's own values, best-response dynamics, an HPR whose objective is penalized by the follower's, PADM,[49] moves of continuous linking variables into the interior, value-function cuts[53,54] and no-good cuts. None of these is needed for correctness. What is certified is each returned point, checked again from scratch by `bobi verify --tight` and, with proven dual bounds on the follower's problem and on the optimistic tie-break, by `bobi certify --tight` (Table 4): the points are solver-certified (class (ii)). `bobi certify --tight` passes 62 of the 64 points without coupling rows and all 29 with them. For the other 2, neos-3627168-kasai_0_100 and neos-3754480-nidda_0_100, the follower's problem at the file's leader values is infeasible at that check's feasibility tolerance of 10^{-9}, so it gets no follower bound; their rows hold within 10^{-6}, and they pass `bobi verify --tight` with the follower solved at HiGHS's default feasibility tolerance of 10^{-7}.

3.7 Formal Verification and Separate Checks

The main lemmas behind the new exact search and the clique bounds are proved in the Lean 4 theorem prover[76] with its mathematical library Mathlib[77] (Table 7). The project contains no `sorry`, and every main theorem uses only Lean's standard axioms (`propext`, `Classical.choice` and `Quot.sound`). The Lean statements are about abstract models (items, weight classes, graphs), not about the Rust code: that the code computes the formalized recursions is argued in its documentation and tested against brute force, not proved. The full map from results to Lean theorems and to the code that relies on them is in Supp. Sec. G.

Separate checks. The software, the experiments and the checks below were carried out with an AI coding agent directed by the author (see the Acknowledgements). The checks described as separate reviews ran in fresh sessions of the same agent that wrote their own code and their own instance readers, without reading the code they checked; they are separate re-implementations, not independent human reviews. The 16 subset-sum lower bounds were proved again by two Python provers that share no code with the Rust search: one written with the search, the other in a separate review session. Both parse the instance files themselves and use only the direct rule and the per-node rule, in integer arithmetic. Two other designs of the search, built separately, prove interKP-200-100-6-3, and every search binary was compared with exhaustive enumeration on 2,990 random small instances, with 0 mismatches. The clique, network and coupled-row results were each derived again in a separate review session with its own instance reader and different solvers or code: two exact maximum-clique searches, HiGHS and a branch and bound in C for the part-size cases; a C implementation of the tree DP compared with ours at every attack; and a second reading of the coupled files with HiGHS's own reader, which also confirmed the

Table 8. Methods, the instances they were run on, their limits per instance, and their outcomes (Supp. Sec. H and the `README` files of the data package give the commands).

Method	Run on	Limits	Outcome
Robust subset-sum search	the 16 open KIP instances; 36 with published optima	none reached; every run finished	16 optimal; all 36 published optima reproduced
LP bracket, merged-row bound, local search	270 OR, one follower knapsack	MILP 60 s (bracket re-run 120 s), local search 30 s	270 points, 191 better lower bounds
Follower LP at the bracket's minimizer	540 OR, several follower knapsacks	MILP 30 s, follower 60 s	420 points
Partitions, part-size cases, tree certificates	60 clique	part-size cases run until decided	57 optimal, 3 open
Enumeration with tree DP	2 feasible and 34 optimal trees	complete; 80 s and 70 s for the two	2 published optima re-proved; 34 values reproduced
Recession directions	general instances	LP, then exact check	40 without a feasible point
HPR, follower, tie-break (`bobi general`)	the 98 open ones without coupling rows	HPR 300 s, follower 300 s	64 points; 17 rays, 10 without an HPR point, 6 follower unsolved, 1 failed check
Portfolio (`bobi coupled`)	54 of the 87 open ones with coupling rows (23 rays and 10 with over 100,000 variables left out)	600 s or 300 s, follower 120 s, batch budget 2.5 h	52 attempted: 29 points, 23 none
Tight verification (`bobi verify --tight`)	all 1,478 solution files	follower MILP 1,200 s	all pass, after 17 + 9 files were re-derived

re-derived ones. The reviews' notes and their corrections are in the `README` files of the data package; Section 5.2 discusses the risk of errors that all these checks share.

4. RESULTS

4.1 Computational Setup

The computations ran on a virtual machine with four cores of an Intel Xeon processor and 15 GB of memory (Ubuntu 24.04; Rust 1.97; Python 3.13; HiGHS 1.15, SCIP 10.0 through PySCIPOpt 6.3, CP-SAT 9.15; Lean 4.35 with Mathlib). The machine was shared with other jobs and often oversubscribed. Every method ran on one thread, except CP-SAT, which used two workers. Table 8 lists the runs. An unsolved instance outside them was not attempted; in particular the general instances that BOBILib lists as feasible were only compared with the literature and, where their point meets the MIPLIB optimum, with the MIPLIB bound. Times are CPU seconds where so stated and wall-clock seconds otherwise, and no proof depends on them. The total computing effort was not recorded; the data package gives per-instance times where measured.

4.2 Overview

Table 2 shows the outcome by family. All 392 unsolved KIP instances are now optimal. Of the 140 MKIP instances that get a verified point where BOBILib lists none, 88 get the first point anywhere; for the other 52 Lozano, Bergman and Cire had published a point with a solution file. The 582 unchanged MKIP instances include many with better points now (Section 4.4). Of the 88 general instances that get a verified point, 79 get the first point anywhere. Of the 79 new optimality proofs, 18 confirm a value BOBILib already listed (the 16 KIP instances and the 2 noswot instances): there the proof is new, not the value. Of the 119 instances settled here, 113 have exact certificates and 6 are optimal at the MIPLIB bound; every new point is solver-certified (two of them, neos-3627168-kasai_0_100 and neos-3754480-nidda_0_100, only by `bobi verify --tight`; Section 3.6), and none is benchmark-accepted (Table 5).

4.3 Knapsack Interdiction

All 16 KIP instances without a published optimum are proven optimal (Table 9). In every case BOBILib's value was already optimal: it equals the capacity c, or $c-1$ in class 7, and every leader choice within budget attains it. The new

Table 9. The 16 knapsack interdiction instances without a published optimum. Class 6: subset sum ($p = w$); class 7: even weights, odd capacity. I: the instance number of the name, which sets the capacity (Section 2.2); β: the leader's budget; c: the follower's capacity. Every optimum equals c (class 6) or $c-1$ (class 7), the trivial upper bound, and BOBILib's value. Nodes and CPU seconds of the search on one thread; prover: nodes of the second prover.

Instance	Items	Class	I	β	c	Optimum	Nodes	CPU (s)	Prover nodes
interKP-200-100-6-2	200	6	2	1,860	1,855	1,855	303	0.10	303
interKP-300-100-6-2	300	6	2	2,865	2,868	2,868	366	0.16	366
interKP-400-100-6-2	400	6	2	3,828	3,832	3,832	257	0.15	257
interKP-500-100-6-2	500	6	2	4,645	4,651	4,651	190	0.26	190
interKP-200-100-6-3	200	6	3	2,787	2,782	2,782	13,498	3.93	13,502
interKP-300-100-6-3	300	6	3	4,299	4,302	4,302	19,405	8.06	19,537
interKP-400-100-6-3	400	6	3	5,744	5,748	5,748	13,430	9.01	13,583
interKP-500-100-6-3	500	6	3	6,970	6,976	6,976	10,079	10.04	10,116
interKP-200-100-7-2	200	7	2	1,878	1,873	1,872	22	0.06	22
interKP-300-100-7-2	300	7	2	2,892	2,895	2,894	27	0.04	27
interKP-400-100-7-2	400	7	2	3,863	3,867	3,866	18	0.05	18
interKP-500-100-7-2	500	7	2	4,689	4,695	4,694	1	0.05	1
interKP-200-100-7-3	200	7	3	2,814	2,809	2,808	651	0.10	651
interKP-300-100-7-3	300	7	3	4,338	4,341	4,340	470	0.20	470
interKP-400-100-7-3	400	7	3	5,797	5,801	5,800	378	0.17	378
interKP-500-100-7-3	500	7	3	7,037	7,043	7,042	256	0.15	256

result is the lower bound. The 12 instances other than those of class 6 with $I = 3$ take at most 651 nodes and 0.26 seconds. Those of class 6 with $I = 3$ are the hardest, with up to 19,405 nodes; the slowest, interKP-500-100-6-3 (optimum 6,976), takes 10,079 nodes and 10.0 seconds. All 16 together take 32.5 seconds. As a check on instances with known answers, the search and the prover written with it reproduce Weninger and Fukasawa's optimum on all 36 class 6 and 7 instances with 100 or 200 items that have one, most of them with optima far below c.

Figure 14 compares the search before and after the per-node tables; "before" is the version immediately preceding the per-node tables, run on the same machine. On interKP-200-100-6-2 it needed 3,926,223 nodes and 22.8 seconds, against 303 nodes and 0.10 seconds now, and it did not finish interKP-200-100-6-3 in a run of 4 hours. The second prover, which uses only the direct and per-node rules, needs almost the same number of nodes. Weninger and Fukasawa's solver was not run again here: it needs Gurobi, which is not available on this machine, and their published runs (16 threads, one hour) already leave these instances open; Weninger's 2026 preprint still reports 16 of the 100 subset-sum instances of Fischetti et al. unsolved after one hour.[25] The current version of their code adds a symmetry-breaking rule that, by its documentation, speeds up subset-sum instances by about an order of magnitude; no results of it are published, and it was not run here.[19] The times are therefore not a like-for-like comparison with their solver.

4.4 The OR Set

Table 10 and Figure 15 show the relative optimality gap $(U - L)/U$, where L and U are the best lower and upper bounds from the literature and this work, over the instances with both a published lower and a published upper bound. The upper bound may be an unverified published point or the follower's LP bound at this work's leader choice. The published points come from Lozano, Bergman and Cire[21,38] and from Lefebvre and Schmidt,[42] whose values count only from runs that ended before the time limit (Section 3.1). For 21 instances a value from one of their runs stopped at the limit would otherwise be the best published point, and for 10 of them it lies below Lozano, Bergman and Cire's lower bound (Supp. Table S6). Of these values, 9 are lower than this work's point on an instance that is counted below among the 474 with a better point: 3 lie below a proven lower bound, and 6 are ruled out by no known bound (for 3 of these the authors' own check file marks the instance as not bilevel feasible), so for those 6 the count rests on the rule of Section 3.1. For one instance, OR10x100-0.75_4.dat-0, a published lower bound exceeds a published point; it is closed (below) and left out of the medians. With one follower knapsack the median gap falls from 24.0% to 4.0% at budget 25% and from 34.0% to 7.1% at budget 50%; at budget 75% the published bounds were already tight. With several follower knapsacks the lower bounds are the literature's, so the gaps move only through better points: little at budget 25%, much more at 75%.

Of the 810 OR instances, 690 now have a verified point: all 270 with one follower knapsack, all 360 with several follower knapsacks at budget 50% or 75%, and 60 of the 180 at budget 25% (Figure 16). Compared with the best published

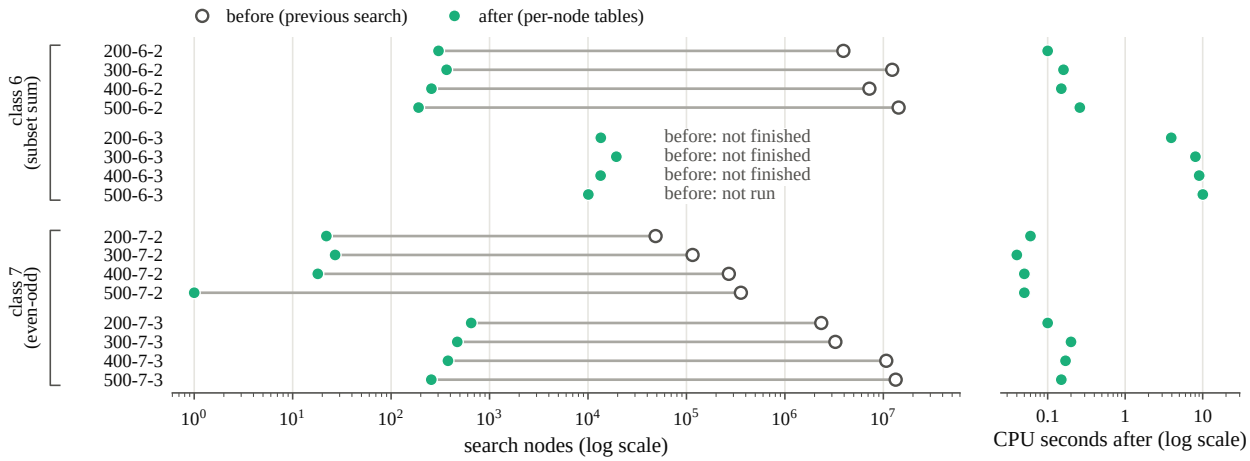


Figure 14. Search nodes for the 16 knapsack interdiction instances open in the literature, before and after the per-node tables (left, log scale), and CPU seconds after (right, log scale). Labels: items-class-*I* of `interKP-`*items*`-100-`*class*`-`*I*. All are proven optimal, the slowest (interKP-500-100-6-3, optimum 6,976) in 10,079 nodes and 10.0 seconds.

Table 10. Median relative gap $(U-L)/U$ on the OR set over the instances with both a published lower and upper bound (paired; OR10x100-0.75_4.dat-0, whose published bounds conflict, left out), with the literature alone and with this work added, and the instances without any published point.

			Median gap (%)		No published point	
Follower knapsacks	Budget	Paired	Literature	Now	Instances	First point here
One (`-100`)	25%	90	24.0	4.0	0	0
	50%	90	34.0	7.1	0	0
	75%	90	0.14	0.14	0	0
Several (`-0`)	25%	55	33.3	31.4	31	0
	50%	59	36.6	29.1	31	31
	75%	62	5.2	0.05	27	27
Several (`-50`)	25%	70	30.1	28.2	20	0
	50%	70	41.3	34.9	20	20
	75%	80	11.8	1.4	10	10

point, BOBILib's included, 474 of these points are strictly better, 72 are equal and 56 are worse; for 88 instances the point is the first anywhere. For 52 more it is the first in BOBILib, although Lozano, Bergman and Cire had published one. With one follower knapsack the two lower bounds of Section 3.3 beat the published one for 191 instances: 166 come from the merged-row online bound (class (i)) and 25 from the LP bracket (class (ii)).

One published optimum needs a note. For OR10x100-0.75_4.dat-0 Lozano, Bergman and Cire published a point of value 8,813 and, in their paper's table, a lower bound of 8,814 from runs they flag as numerically inconsistent; their repository also records a branch-and-cut bound of 8,813.[38] The point and the branch-and-cut bound prove the value optimal, so the instance is counted among the published optima. The verified solution file of this work at 8,813 (checked also with SCIP) confirms that the bound 8,814 is wrong.

4.5 Clique Interdiction

Of the 60 clique instances, 57 are optimal (Figure 17): 27 at density 0.7 with value 3 and all 30 at density 0.9 with value 4, where BOBILib lists 5 to 13. All 57 rest on exact certificates (Section 3.4); BOBILib's points are valid, but their leader choices are far from optimal. The 3 open instances, normalClique60-70-2, normalClique60-70-3 and normalClique60-70-5, have value 3 or 4; both bounds are exact (Mantel's theorem and a 4-partition). Value 3 needs a set of at most $k = 309$ edges that meets every K_4. The best 3-partitions found have 311, 310 and 310 inside edges, and searches that are not restricted to 3-partite remainders stop at the same values. A proof of value 4 cannot use Brouwer's theorem: $m-k = 930$ is far below its threshold 1,182, and there are K_4-free graphs with that many edges that are not 3-partite (Supp. Sec. D.6). LP bounds on the smallest such edge set stop near 296.

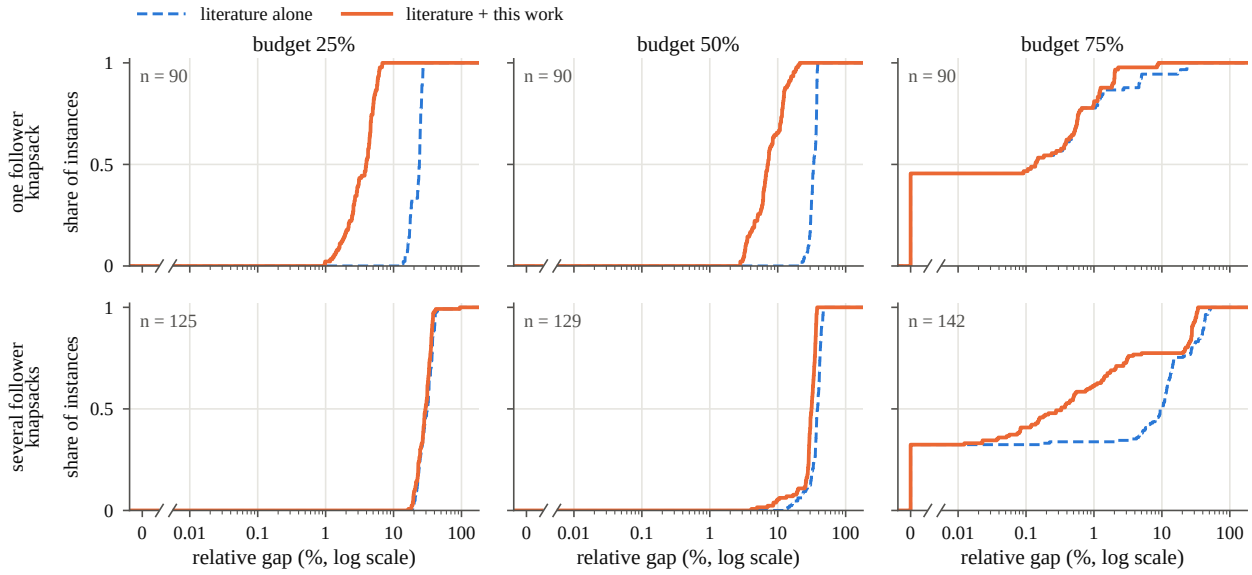


Figure 15. Relative optimality gap $(U - L)/U$ on the OR set over the instances with both published bounds (OR10x100-0.75_4.dat-0, whose published bounds conflict, left out), literature alone against literature plus this work, by follower type and budget (cumulative share; n: instances in the panel). Gaps of 0, closed instances, are drawn at the separate "0" tick; no gap is negative.

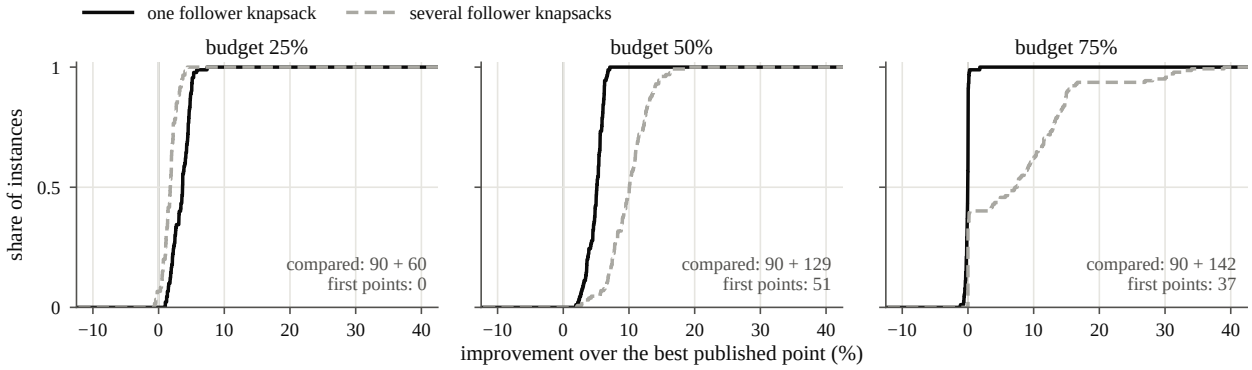


Figure 16. Improvement of the verified OR points over the best published point (literature or BOBILib), by budget and follower type (cumulative share). In each panel, "compared" gives the points compared with one and with several follower knapsacks, "first points" the instances with no published point (not shown), and "left of −10%" the points off the scale on the left; the point of OR10x100-0.75_4.dat-0, whose published bounds conflict, is left out. Of the 690 points, 474 are strictly better.

4.6 Network and General Instances

The network instances tree-50_1-5-5_006 and tree-50_1-5-5_010 are optimal at 26, BOBILib's value, as Fischetti et al. had proved with MIX++;[48] the enumeration confirms it in exact arithmetic: 4 and 28 of the 2,138,410 attacks reach it. Of BOBILib's 70 optimal network values, 54 are derived again: the 34 trees by the same code, the 12 random graphs with 25 nodes by HiGHS at every attack, and 8 random graphs with 50 or 80 nodes by the brute force of the separate review.

Figure 18 shows the outcome for the 266 unsolved general instances. Of the instances that BOBILib lists as open, 40 have no bilevel feasible point, all with exact certificates. BOBILib's paper notes that an instance without coupling rows whose MILP relaxation is feasible is "technically" feasible even when no point is known;[11] that needs a follower with an optimal response, and the 17 _0_100 instances among the 40 are counterexamples. The LP test of Section 3.6 detects all 40, and most of them have continuous linking variables, which BOBILib's two solvers do not handle. Without coupling rows, `bobi general` was run on 98 open instances. Of these, 17 have no bilevel feasible point, and of the other 81 it gives verified points for 64, 4 of them optimal (below). For 8 of the other 60 a point had been published elsewhere, better than ours for 6 (Supp. Table S12); for the rest it is the first point known. With coupling rows, `bobi coupled` was run on 52 of 54 candidates and found 29 verified points: 21 at the HPR's own leader values, 5 from the penalized HPR and 3 from

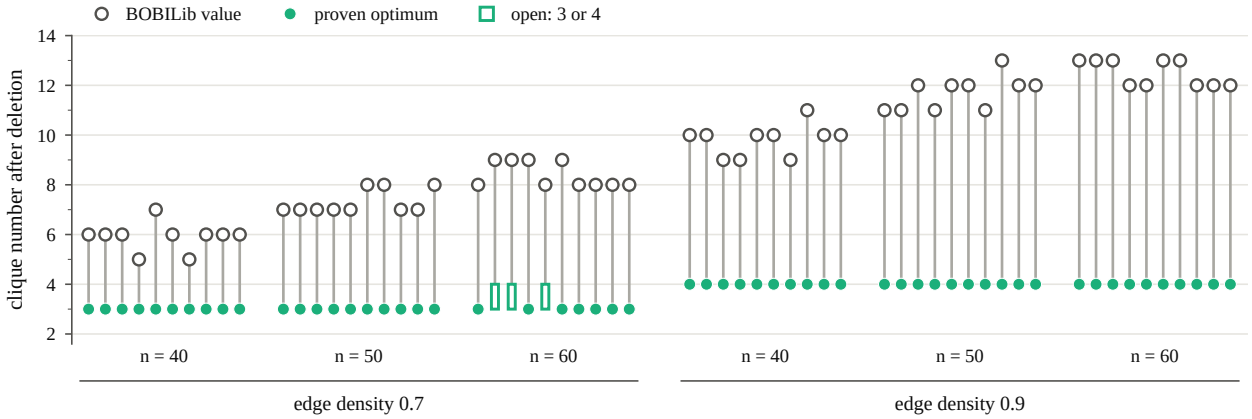


Figure 17. The 60 clique interdiction instances: BOBILib's value (5 to 13, open circle) against the proven optimum (3 at density 0.7, 4 at density 0.9). The 3 open instances lie between 3 and 4.

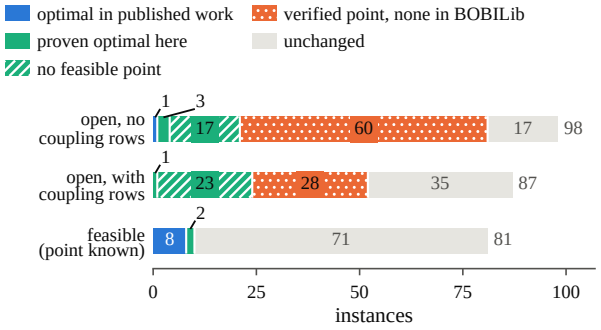


Figure 18. Outcomes for the 266 general instances BOBILib lists as feasible or open, by BOBILib status and, for the open ones, by whether leader rows contain follower variables (coupling rows).

Table 11. Instances whose HPR is exactly the MIPLIB model they were built from, with a verified point at its published optimum (class (ii)): the 6 settled here and, marked *, a published optimum that this confirms. BOBILib: status before this work; point from: BOBILib's solution file or the method of this work. MIPLIB's optimum of noswot, −41.00000885, rounds up because its objective takes integer values only.

Instance	BOBILib	Optimum	Point from
noswot_0_100	feasible	−41	BOBILib
noswot_50_50	feasible	−41	BOBILib
pigeon-10_0_100	open	−9,000	`bobi general`
timtab1_0_100	open	764,771.99999978	`bobi general`
pigeon-10_50_50	open	−9,000	`bobi coupled`
ex10_0_100*	open	100	`bobi general`
neos-1445765_0_100	open	−17,783	`bobi general`

best-response dynamics. All these points pass the tight test, 17 without and 9 with coupling rows after the re-derivation of Section 3.1.

Table 11 lists the 6 instances proven optimal here at the MIPLIB bound; 4 of them were open in BOBILib, without a BOBILib point. It also lists ex10_0_100, which BOBILib lists as open and whose optimum 100 Lefebvre and Schmidt's run files had reported, from a run that ended before the time limit (no solution file is published; Section 3.1): the MIPLIB bound and the verified point of `bobi general` confirm this published optimum independently. For noswot_0_100 and noswot_50_50 BOBILib's own points already had the optimal value; the proof is new. For noswot, ex10, neos-1445765 and pigeon-10 the objective takes integer values only (for pigeon-10 multiples of 1,000), so the results are safe by integrality rounding if MIPLIB's bounds are valid; for timtab1_0_100 point and bound agree numerically (Section 3.6). The generalized interdiction instances that remain unchanged (8 of them) have many linking variables: enumerating their binary linking choices, which reproduces a published optimum on a smaller instance of the set, is too slow for them.

5. DISCUSSION

The new results come from structure, certificates and published bounds that a general-purpose MIBLP solver does not use. A general branch and bound over the 1,593 edge variables of a 60-vertex clique instance at density 0.9 is unlikely to show quickly that every deletion of 398 edges leaves a K_4, while Brouwer's theorem and 170 small cases do; and an instance without a bilevel feasible point needs one direction as a certificate, which one LP finds,[27] while a solver that only searches for a point can only report a time limit.

5.1 Recommendations for Benchmark Libraries

Benchmark statuses are most useful when they are current and when each comes with its evidence. From this work I suggest:

- record the provenance of every status (a solver run with its limits, a published proof, or a certificate) and its certificate class, and import published per-instance results, so that an instance solved elsewhere is not listed as unsolved;
- screen new instances for followers that are infeasible or unbounded for every leader choice (for unboundedness one LP suffices[27]), and accept certificates of bilevel infeasibility, such as the recession directions here, as evidence for an infeasible status; Rodrigues et al. make a similar case for checks and certificates of unboundedness in bilevel optimization;[78]
- state the tolerances behind every status, and prefer exactly optimal follower responses. A relative test such as (3) lets the leader profit from a suboptimal follower when the follower's values are large, as in BOBILib's file for general20-30-20-20-20-2 (Section 3.1), which `bobi verify` rejects: it misses the rows by more than 10^{-6}, and its stated objective 100,056 differs from the 100,056.207 of its point (Supp. Sec. E.5). Published results have it too, as Lozano, Bergman and Cire's decision-diagram value for p0548-0.900000 shows (Section 3.1), and this work had it in 25 of its own earlier files, removed with a tight test;
- distribute solution files and certificates in standard formats, with a status file in the MIPLIB `.solu` format, as the data package of this paper does.

5.2 Limitations and Threats to Validity

Running times. All times come from a shared four-core machine that was often oversubscribed; they show orders of magnitude, not careful benchmarks.

Solver-certified results. The results of class (ii) trust floating-point solvers: follower optimality for followers with several rows and for general instances, the dual bounds of the LP bracket and the MIPLIB optima behind the 6 bound-based optima, which are published floating-point values (timtab1_0_100, for example, is optimal at 764,771.99999978, MIPLIB's published value). The points are feasible within 10^{-6} and follower-optimal within the tight test, not exactly; for neos-3627168-kasai_0_100 and neos-3754480-nidda_0_100 the check with proven dual bounds gets no follower bound, and they rest on `bobi verify --tight` alone. Certificates in a checkable proof format, such as VIPR for MILP verdicts[79] from an exact rational solver[80] or pseudo-Boolean proof logs checked by VeriPB,[81] would remove this trust; they were not produced here. The window rule of the subset-sum search uses a floating-point fractional bound; the 16 subset-sum proofs are confirmed by the integer-only provers, which do not use that rule.

Exact certificates rely on code. The exact checks run tested but unverified code: the rational checks of the recession directions and of the MIPLIB comparison, the clique checker, the network enumeration and the subset-sum provers. Each is short or was written twice, but none is proved correct.

Scope of the Lean proofs. Lean covers the main pruning lemmas of the KIP search and the Mantel and partition lemmas for clique interdiction. It does not cover the reduction to the count formulation, the cheaper rules of Supp. Thm B.23, Brouwer's theorem, the LP bracket, the network DP, the recession directions, or the search code itself (Table 7).

Results taken from the literature. The 531 published optima, the work of their authors, the published lower bounds of the OR instances with several follower knapsacks, and Brouwer's theorem are used as published; for 82 of the optima not even a point is verified here. Lefebvre and Schmidt's values are taken only from runs that ended before the time limit, as in most of their own analysis; the one optimum taken from their run files is confirmed by a verified point and the MIPLIB bound. "First point anywhere" means that none of the published sources compiled here (Supp. Sec. H) has a point; for 9 of the general instances that get a feasible point from `bobi general` or `bobi coupled` a published point exists (for 3 of them only in Lefebvre and Schmidt's run files), and these are not counted as first points; ex10_0_100, whose optimum is taken from their run files, is counted as a published optimum (Section 3.1). Published points that beat BOBILib's value on instances whose status this work leaves unchanged were not compiled systematically: Fischetti et al.'s Table 6 has such points for several MIPLIB 3 instances, and for air04-0.900000, which BOBILib lists as open, a point where BOBILib has none;[15] Lefebvre and Schmidt's run files have such points for some other instances; and other sources may hold points that this reconciliation missed.

Common-mode errors. The solvers, the verifier and the separate review sessions were all written with the same AI coding agent directed by the same author. Separate sessions with their own code and instance readers reduce, but do not remove, the risk of a shared misreading of the file format or a shared conceptual slip. The Lean proofs, the exact rational and integer checks and the brute-force comparisons are the checks least exposed to it.

Instance reading. Every argument depends on reading the instance files correctly. Each family's model was checked row by row against its expected pattern, and the key results were derived again with separately written readers.

5.3 What Stays Hard

For the 3 open clique instances, value 3 needs a set of at most k edges that meets every K_4, which no search has found, and value 4 needs a lower bound on the size of such a set well above the LP bounds. For random graphs the largest K_4-free subgraphs are 3-partite,[82,83] which would point to value 4, but these results are asymptotic and give no bound for a given graph with 60 vertices. The OR instances with several follower knapsacks at budget 25% still have no certified point in 120 cases: 51 have no point anywhere, and 69 have only a published, unverified point. Their median gaps stay near 31.4% and 28.2%; they need a lower bound that uses the structure of several knapsacks, which the one-row bracket does not. Generalized interdiction with many linking variables needs a pruned enumeration of the linking choices, and 123 general instances (122 of them MIPLIB-derived) remain without a point or without a proof. The results of this work will be submitted to BOBILib's maintainers, who update the statuses from submitted solutions.

6. CONCLUSIONS

I asked which of BOBILib's 1,594 feasible or open instances can be settled. The key findings are:

- Published work had already solved 531 of them (among them the two network instances, which exhaustive enumeration with an exact tree program re-proves here); reconciling BOBILib's statuses, which predate some of these results, with the literature settles more instances than any single method.
- Structure-specific methods settle 119 more, 113 of them with exact certificates: a robust subset-sum search proves all 16 remaining knapsack interdiction instances optimal, by computational proofs supported by formally verified lemmas; extremal graph theory, with partition certificates and 170 integer-checked case certificates, settles 57 of 60 clique interdiction instances; recession directions, all checked in exact arithmetic, show that 40 instances have no bilevel feasible point; and published MIPLIB optima make the remaining 6 of the 79 new optima optimal by the conventions of MIPLIB and BOBILib (solver-certified, class (ii)).
- In all, 650 instances (41%) are settled. Another 228 get a verified point where BOBILib lists none, 167 of them the first point anywhere, and 474 OR instances get a point better than any published one.
- The claims can be checked: every settled instance and every new point carries a certificate class; all 1,478 current solution files are in BOBILib's format and pass a tight test of follower optimality, so none relies on BOBILib's tolerance; the certificates are published; and the main pruning lemmas of the new search are proved in Lean.

DATA AVAILABILITY

The data package is the folder `results/` of the repository https://github.com/brian-j-connolly-aero/bobilib. It contains `instances.csv`, one row per BOBILib instance with BOBILib's status and value, the new status, the best known value (verified here, or a published optimum) and the best bound, the source, the method, the certificate and its class; `bobilib_update.solu`, the status changes in MIPLIB's `.solu` format (610 `=opt=`, 40 `=inf=`, 693 `=best=` and 194 `=bestdual=` lines); the solution files in BOBILib's format, with the superseded files of Section 3.1; the certificates (recession directions, clique partitions and case trees with their checker and the solver logs of the part-size cases, subset-sum proof records, enumeration summaries, OR bounds); notes on the tight verification, the instance structure and the comparison with Lefebvre and Schmidt's run files; and `SHA256SUMS`, the checksums of every file. The code is in the same repository: the Rust crate `bobi` (the solvers and the verifier), the scripts of the separate checks, and the Lean project with the proofs. The supplementary document (`paper/appendix/` in the repository) holds the proofs, the verification protocol, the certificate classes, the map of the Lean theorems and the per-instance tables.

DISCLOSURE OF AI ASSISTANCE

This work was carried out with substantial assistance from an artificial intelligence (AI) coding agent, Claude Code (Anthropic), working under my direction. I posed the question, set the priorities, chose which directions to pursue, reviewed the results and the manuscript, and decided what to claim. The agent did most of the hands-on work:

- the literature search and the reconciliation of BOBILib's statuses with published per-instance data;
- all software: the Rust solvers and the verifier (`bobi`), the scripts, the checkers and the data package, and the computational experiments;
- the design of the new methods, including the per-node tables of the subset-sum search, which came out of a comparison of three designs built and judged in separate agent sessions;
- the Lean formalisation of the pruning rules and the clique lemmas;
- the figures, the first drafts of this paper and of the supplementary document, and their revisions.

The checks called separate reviews in Section 3.7 were fresh sessions of the same agent that wrote their own code and instance readers without reading the code they checked; they are re-implementations, not independent human reviews, and they share the risk of a common misreading (Section 5.2). Comments from another AI system (ChatGPT, OpenAI) on an earlier draft led to the exact certificates for the last two recession directions, the re-derivation of the tolerance-dependent points, the clique case certificates and the certificate classes. Apart from me, no human has yet reviewed the code or the proofs. For that reason every claim comes with an artefact that can be checked without trusting the agent or its code: solution files that BOBILib's own checker can read, certificates with short standalone checkers, Lean proofs checked by Lean's kernel, and the per-instance records of the data package. All results reported here passed the verification described in Sections 3.1 and 3.7.

ACKNOWLEDGEMENTS

I thank the maintainers of BOBILib, Johannes Thürauf, Thomas Kleinert, Ivana Ljubić, Ted Ralphs and Martin Schmidt, for the library, its solution files and its checker, and the authors who published their per-instance data and code, in particular Noah Weninger and Ricardo Fukasawa; Matteo Fischetti, Ivana Ljubić, Michele Monaci and Markus Sinnl; Leonardo Lozano, David Bergman and Andre A. Cire; and Henri Lefebvre and Martin Schmidt.

REFERENCES

[1] von Stackelberg, H., *Market Structure and Equilibrium*, Springer, Berlin (2011). English translation by D. Bazin et al. of *Marktform und Gleichgewicht*, Julius Springer, Vienna, 1934. https://doi.org/10.1007/978-3-642-12586-7.

[2] Brown, G., Carlyle, M., Salmerón, J. and Wood, K., "Defending critical infrastructure," *Interfaces* **36**(6), 530–544 (2006). https://doi.org/10.1287/inte.1060.0252.

[3] Smith, J. C. and Song, Y., "A survey of network interdiction models and algorithms," *European Journal of Operational Research* **283**(3), 797–811 (2020). https://doi.org/10.1016/j.ejor.2019.06.024.

[4] Jeroslow, R. G., "The polynomial hierarchy and a simple model for competitive analysis," *Mathematical Programming* **32**(2), 146–164 (1985). https://doi.org/10.1007/BF01586088.

[5] Caprara, A., Carvalho, M., Lodi, A. and Woeginger, G. J., "A study on the computational complexity of the bilevel knapsack problem," *SIAM Journal on Optimization* **24**(2), 823–838 (2014). https://doi.org/10.1137/130906593.

[6] Dempe, S., *Foundations of Bilevel Programming*, Nonconvex Optimization and Its Applications, Kluwer Academic Publishers (2002). https://doi.org/10.1007/b101970.

[7] Kleinert, T., Labbé, M., Ljubić, I. and Schmidt, M., "A survey on mixed-integer programming techniques in bilevel optimization," *EURO Journal on Computational Optimization* **9**, 100007 (2021). https://doi.org/10.1016/j.ejco.2021.100007.

[8] Bixby, R. E., Ceria, S., McZeal, C. M. and Savelsbergh, M. W. P., "An updated mixed integer programming library: MIPLIB 3.0," *Optima* **58**, 12–15 (1998). https://miplib2010.zib.de/miplib3/miplib.html.

[9] Koch, T., Achterberg, T., Andersen, E., Bastert, O., Berthold, T., Bixby, R. E., Danna, E., Gamrath, G., Gleixner, A. M., Heinz, S., Lodi, A., Mittelmann, H., Ralphs, T., Salvagnin, D., Steffy, D. E. and Wolter, K., "MIPLIB 2010," *Mathematical Programming Computation* **3**(2), 103–163 (2011). https://doi.org/10.1007/s12532-011-0025-9.

[10] Gleixner, A., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., Christophel, P., Jarck, K., Koch, T., Linderoth, J., Lübbecke, M., Mittelmann, H. D., Ozyurt, D., Ralphs, T. K., Salvagnin, D. and Shinano, Y., "MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library," *Mathematical Programming Computation* **13**(3), 443–490 (2021). https://doi.org/10.1007/s12532-020-00194-3.

[11] Thürauf, J., Kleinert, T., Ljubić, I., Ralphs, T. K. and Schmidt, M., "BOBILib: Bilevel Optimization (Benchmark) Instance Library," *Mathematical Programming Computation* **18**(3), 819–876 (2026). https://doi.org/10.1007/s12532-025-00294-y.

[12] Thürauf, J., Kleinert, T., Ljubić, I., Ralphs, T. K. and Schmidt, M., "BOBILib: Bilevel Optimization (Benchmark) Instance Library," Website with instances, solution files, statuses and feasibility checker (2026). Accessed 18 October 2026. https://bobilib.org.

[13] Tahernejad, S., Ralphs, T. K. and DeNegre, S. T., "A branch-and-cut algorithm for mixed integer bilevel linear optimization problems and its implementation," *Mathematical Programming Computation* **12**(4), 529–568 (2020). https://doi.org/10.1007/s12532-020-00183-6.

[14] DeNegre, S. T., Ralphs, T. K., Tahernejad, S. and Xie, Y., "MibS," Open-source solver for mixed integer bilevel linear optimization, COIN-OR; BOBILib's statuses use version 1.2.2 (2024). https://github.com/coin-or/MibS.

[15] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "A new general-purpose algorithm for mixed-integer bilevel linear programs," *Operations Research* **65**(6), 1615–1637 (2017). https://doi.org/10.1287/opre.2017.1650.

[16] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Bilevel integer programming and interdiction problems," Web page with the intersection-cut solver for mixed-integer bilevel linear problems (MIX++ setting) and instance sets (2024). Accessed 18 October 2026. https://msinnl.github.io/pages/bilevel.html.

[17] Caprara, A., Carvalho, M., Lodi, A. and Woeginger, G. J., "Bilevel knapsack with interdiction constraints," *INFORMS Journal on Computing* **28**(2), 319–333 (2016). https://doi.org/10.1287/ijoc.2015.0676.

[18] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Interdiction games and monotonicity," technical report, DEI, University of Padova (2016). Preprint of the 2019 article in INFORMS J. Comput.; PDF of 20 December 2016, an earlier version of 30 August 2016 at https://msinnl.github.io/pdfs/independentSystems-techreport.pdf. Accessed 28 October 2026. https://www.dei.unipd.it/~fisch/papers/interdiction_games_and_monotonicity.pdf.

[19] Weninger, N. and Fukasawa, R., "bkpsolver: code accompanying 'A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints'," Source code, instances and answer files (MIT license) (2022). Initial commit d8c3b74 with the instances and answer files, public by 7 October 2022; commits 1838efe (4 May 2023, counted copies) and bc1d985 (3 August 2026, symmetry breaking by item dominance). https://github.com/nwoeanhinnogaehr/bkpsolver.

[20] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Interdiction games and monotonicity, with application to knapsack problems," *INFORMS Journal on Computing* **31**(2), 390–410 (2019). https://doi.org/10.1287/ijoc.2018.0831.

[21] Lozano, L., Bergman, D. and Cire, A. A., "Network relaxations for combinatorial bilevel optimization under linear interactions," *Operations Research* (2026). Articles in Advance, published online 26 August 2026. https://doi.org/10.1287/opre.2025.2457.

[22] Wood, R. K., "Deterministic network interdiction," *Mathematical and Computer Modelling* **17**(2), 1–18 (1993). https://doi.org/10.1016/0895-7177(93)90236-R.

[23] Israeli, E. and Wood, R. K., "Shortest-path network interdiction," *Networks* **40**(2), 97–111 (2002). https://doi.org/10.1002/net.10039.

[24] Kellerer, H., Pferschy, U. and Pisinger, D., *Knapsack Problems*, Springer, Berlin (2004). https://doi.org/10.1007/978-3-540-24777-7.

[25] Weninger, N., "A simple $(2+\varepsilon)$-approximation for knapsack interdiction," arXiv preprint 2604.21877 (2026). Version 2 of 16 October 2026 (version 1: 23 April 2026). https://doi.org/10.48550/arXiv.2604.21877.

[26] Weninger, N. and Fukasawa, R., "A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints," *Mathematical Programming* **210**(1–2), 847–879 (2025). https://doi.org/10.1007/s10107-024-02133-9.

[27] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "On the use of intersection cuts for bilevel optimization," *Mathematical Programming* **172**(1–2), 77–103 (2018). https://doi.org/10.1007/s10107-017-1189-5.

[28] Xu, P. and Wang, L., "An exact algorithm for the bilevel mixed integer linear programming problem under three simplifying assumptions," *Computers & Operations Research* **41**, 309–318 (2014). https://doi.org/10.1016/j.cor.2013.07.016.

[29] Meyer, R. R., "On the existence of optimal solutions to integer and mixed-integer programming problems," *Mathematical Programming* **7**(1), 223–235 (1974). https://doi.org/10.1007/BF01585518.

[30] Furini, F., Ljubić, I., San Segundo, P. and Zhao, Y., "A branch-and-cut algorithm for the Edge Interdiction Clique Problem," *European Journal of Operational Research* **294**(1), 54–69 (2021). https://doi.org/10.1016/j.ejor.2021.01.030.

[31] Zhou, Y., Jiang, H., Zhu, C. and Rossi, A., "Exact clique number manipulation via edge interdiction," *European Journal of Operational Research* (2026). In press; preprint arXiv 2601.01869 (5 January 2026). https://doi.org/10.1016/j.ejor.2026.07.036.

[32] Moore, J. T. and Bard, J. F., "The mixed integer linear bilevel programming problem," *Operations Research* **38**(5), 911–921 (1990). https://doi.org/10.1287/opre.38.5.911.

[33] Fischetti, M., Monaci, M. and Sinnl, M., "A dynamic reformulation heuristic for Generalized Interdiction Problems," *European Journal of Operational Research* **267**(1), 40–51 (2018). https://doi.org/10.1016/j.ejor.2017.11.043.

[34] Martello, S., Pisinger, D. and Toth, P., "Dynamic programming and strong bounds for the 0-1 knapsack problem," *Management Science* **45**(3), 414–424 (1999). https://doi.org/10.1287/mnsc.45.3.414.

[35] Della Croce, F. and Scatamacchia, R., "An exact approach for the bilevel knapsack problem with interdiction constraints and extensions," *Mathematical Programming* **183**(1–2), 249–281 (2020). https://doi.org/10.1007/s10107-020-01482-5.

[36] DeNegre, S., *Interdiction and Discrete Bilevel Linear Programming*. PhD thesis, Lehigh University, Bethlehem, PA (2011). https://coral.ise.lehigh.edu/~ted/files/papers/ScottDeNegreDissertation11.pdf.

[37] Weninger, N. and Fukasawa, R., "A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints," in *Integer Programming and Combinatorial Optimization (IPCO 2023)*, Del Pia, A. and Kaibel, V., eds., *Lecture Notes in Computer Science* **13904**, 438–452, Springer, Cham (2023). https://doi.org/10.1007/978-3-031-32726-1_31.

[38] Lozano, L., Bergman, D. and Cire, A. A., "DiscreteBilevelOptimization: code, computational results and BOBILib solution files for 'network relaxations for combinatorial bilevel optimization under linear interactions'," Source code and data (2026). https://github.com/andrecire/DiscreteBilevelOptimization.

[39] Chu, P. C. and Beasley, J. E., "A genetic algorithm for the multidimensional knapsack problem," *Journal of Heuristics* **4**(1), 63–86 (1998). https://doi.org/10.1023/A:1009642405419.

[40] Beasley, J. E., "OR-Library: distributing test problems by electronic mail," *Journal of the Operational Research Society* **41**(11), 1069–1072 (1990). https://doi.org/10.1057/jors.1990.166.

[41] Khuri, S., Bäck, T. and Heitkötter, J., "The zero/one multiple knapsack problem and genetic algorithms," in *Proceedings of the 1994 ACM Symposium on Applied Computing (SAC '94)*, 188–193, ACM, New York, NY (1994). https://doi.org/10.1145/326619.326694.

[42] Lefebvre, H. and Schmidt, M., "A Dantzig–Wolfe single-level reformulation for mixed-integer linear bilevel optimization: exact and heuristic approaches," preprint, Optimization Online (June 2025). Per-instance results on the first author's web page, henrilefebvre.com. https://optimization-online.org/2025/06/a-dantzig-wolfe-single-level-reformulation-for-mixed-integer-linear-bilevel-optimization-exact-and-heuristic-approaches/.

[43] Tang, Y., Richard, J.-P. P. and Smith, J. C., "A class of algorithms for mixed-integer bilevel min-max optimization," *Journal of Global Optimization* **66**(2), 225–262 (2016). https://doi.org/10.1007/s10898-015-0274-7.

[44] Furini, F., Ljubić, I., Martin, S. and San Segundo, P., "The maximum clique interdiction problem," *European Journal of Operational Research* **277**(1), 112–127 (2019). https://doi.org/10.1016/j.ejor.2019.02.028.

[45] Hassan, M.-A., Kacem, I., Martin, S. and Osman, I. M., "On the *m*-clique free interval subgraphs polytope: polyhedral analysis and applications," *Journal of Combinatorial Optimization* **36**(3), 1074–1101 (2018). https://doi.org/10.1007/s10878-018-0291-9.

[46] Mahdavi Pajouh, F., "Minimum cost edge blocker clique problem," *Annals of Operations Research* **294**(1–2), 345–376 (2020). https://doi.org/10.1007/s10479-019-03315-x.

[47] Baggio, A., Carvalho, M., Lodi, A. and Tramontani, A., "Multilevel approaches for the critical node problem," *Operations Research* **69**(2), 486–508 (2021). https://doi.org/10.1287/opre.2020.2014.

[48] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Appendix of: A new general-purpose algorithm for mixed-integer bilevel linear programs," Online appendix with detailed computational results, dated 29 June 2016 (2016). Accessed 28 October 2026. http://www.dei.unipd.it/~fisch/appendix.pdf.

[49] Kleinert, T. and Schmidt, M., "Computing feasible points of bilevel problems with a penalty alternating direction method," *INFORMS Journal on Computing* **33**(1), 198–215 (2021). https://doi.org/10.1287/ijoc.2019.0945.

[50] Fischetti, M., Ljubić, I., Monaci, M. and Sinnl, M., "Intersection cuts for bilevel optimization," in *Integer Programming and Combinatorial Optimization (IPCO 2016)*, Louveaux, Q. and Skutella, M., eds., *Lecture Notes in Computer Science* **9682**, 77–88, Springer, Cham (2016). https://doi.org/10.1007/978-3-319-33461-5_7.

[51] Özaltın, O. Y. and Zhang, J., "A branch-and-cut algorithm for discrete bilevel linear programs," tech. rep., Optimization Online (2017). https://optimization-online.org/?p=14558.

[52] DeNegre, S. T. and Ralphs, T. K., "A branch-and-cut algorithm for integer bilevel linear programs," in *Operations Research and Cyber-Infrastructure*, Chinneck, J. W., Kristjansson, B. and Saltzman, M. J., eds., *Operations Research/Computer Science Interfaces* **47**, 65–78, Springer, Boston, MA (2009). https://doi.org/10.1007/978-0-387-88843-9_4.

[53] Mitsos, A., "Global solution of nonlinear mixed-integer bilevel programs," *Journal of Global Optimization* **47**(4), 557–582 (2010). https://doi.org/10.1007/s10898-009-9479-y.

[54] Lozano, L. and Smith, J. C., "A value-function-based exact approach for the bilevel mixed-integer programming problem," *Operations Research* **65**(3), 768–786 (2017). https://doi.org/10.1287/opre.2017.1589.

[55] Vásquez, S., Lozano, L. and van Hoeve, W.-J., "A single-level reformulation of binary bilevel programs using decision diagrams," *Mathematical Programming* (2025). Published online 16 December 2025. https://doi.org/10.1007/s10107-025-02294-1.

[56] Weninger, N. and Fukasawa, R., "A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints," preprint, Optimization Online (Oct. 2022). https://optimization-online.org/2022/10/a-fast-combinatorial-algorithm-for-the-bilevel-knapsack-problem-with-interdiction-constraints/.

[57] Della Croce, F. and Scatamacchia, R., "Lower bounds and a new exact approach for the bilevel knapsack with interdiction constraints," in *Integer Programming and Combinatorial Optimization (IPCO 2019)*, Lodi, A. and Nagarajan, V., eds., *Lecture Notes in Computer Science* **11480**, 155–167, Springer, Cham (2019). https://doi.org/10.1007/978-3-030-17953-3_12.

[58] Dantzig, G. B., "Discrete-variable extremum problems," *Operations Research* **5**(2), 266–288 (1957). https://doi.org/10.1287/opre.5.2.266.

[59] Martello, S. and Toth, P., *Knapsack Problems: Algorithms and Computer Implementations*, Wiley-Interscience Series in Discrete Mathematics and Optimization, John Wiley & Sons, Chichester (1990).

[60] Huangfu, Q. and Hall, J. A. J., "Parallelizing the dual revised simplex method," *Mathematical Programming Computation* **10**(1), 119–142 (2018). https://doi.org/10.1007/s12532-017-0130-5.

[61] Brown, J. L., "Note on complete sequences of integers," *The American Mathematical Monthly* **68**(6), 557–560 (1961). https://doi.org/10.2307/2311150.

[62] Glover, F., "Surrogate constraints," *Operations Research* **16**(4), 741–749 (1968). https://doi.org/10.1287/opre.16.4.741.

[63] Mantel, W., "Problem 28 (solution by H. Gouwentak, W. Mantel, J. Teixeira de Mattes, F. Schuh and W. A. Wythoff)," *Wiskundige Opgaven* **10**, 60–61 (1907).

[64] Brouwer, A. E., "Some lotto numbers from an extension of Turán's theorem," Afdeling Zuivere Wiskunde report ZW 152/81, Mathematisch Centrum, Amsterdam (Mar. 1981). https://ir.cwi.nl/pub/6791.

[65] Tyomkyn, M. and Uzzell, A. J., "Strong Turán stability," *The Electronic Journal of Combinatorics* **22**(3), P3.9 (2015). https://doi.org/10.37236/4717.

[66] Turán, P., "Egy gráfelméleti szélsőértékfeladatról [On an extremal problem in graph theory]," *Matematikai és Fizikai Lapok* **48**, 436–452 (1941). In Hungarian.

[67] Hanson, D. and Toft, B., "k-saturated graphs of chromatic number at least k," *Ars Combinatoria* **31**, 159–164 (1991).

[68] Kang, M. and Pikhurko, O., "Maximum K_{r+1}-free graphs which are not r-partite," *Matematychni Studii* **24**(1), 12–20 (2005).

[69] Amin, K., Faudree, J., Gould, R. J. and Sidorowicz, E., "On the non-$(p-1)$-partite K_p-free graphs," *Discussiones Mathematicae Graph Theory* **33**(1), 9–23 (2013). https://doi.org/10.7151/dmgt.1654.

[70] Perron, L. and Didier, F., "CP-SAT, version 9.15," Google OR-Tools (2026). https://developers.google.com/optimization/cp/cp_solver/.

[71] Hojny, C., Besançon, M., Bestuzheva, K., Borst, S., Chmiela, A., Dionísio, J., Eifler, L., Ghannam, M., Gleixner, A., Göß, A., Hoen, A., van der Hulst, R., Kamp, D., Koch, T., Kofler, K., Lentz, J., Maher, S. J., Mexi, G., Mühmer, E., Pfetsch, M. E., Pokutta, S., Serrano, F., Shinano, Y., Turner, M., Vigerske, S., Walter, M., Weninger, D. and Xu, L., "The SCIP Optimization Suite 10.0," technical report, Optimization Online (Nov. 2025). https://optimization-online.org/2025/11/the-scip-optimization-suite-10-0/.

[72] Maher, S., Miltenberger, M., Pedroso, J. P., Rehfeldt, D., Schwarz, R. and Serrano, F., "PySCIPOpt: mathematical programming in Python with the SCIP Optimization Suite," in *Mathematical Software – ICMS 2016*, *Lecture Notes in Computer Science* **9725**, 301–307, Springer, Cham (2016). https://doi.org/10.1007/978-3-319-42432-3_37.

[73] Bron, C. and Kerbosch, J., "Algorithm 457: finding all cliques of an undirected graph," *Communications of the ACM* **16**(9), 575–577 (1973). https://doi.org/10.1145/362342.362367.

[74] Tomita, E., Tanaka, A. and Takahashi, H., "The worst-case time complexity for generating all maximal cliques and computational experiments," *Theoretical Computer Science* **363**(1), 28–42 (2006). https://doi.org/10.1016/j.tcs.2006.06.015.

[75] Carraghan, R. and Pardalos, P. M., "An exact algorithm for the maximum clique problem," *Operations Research Letters* **9**(6), 375–382 (1990). https://doi.org/10.1016/0167-6377(90)90057-C.

[76] de Moura, L. and Ullrich, S., "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, Platzer, A. and Sutcliffe, G., eds., *Lecture Notes in Computer Science* **12699**, 625–635, Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79876-5_37.

[77] The mathlib Community, "The Lean mathematical library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, 367–381, ACM, New York, NY (2020). https://doi.org/10.1145/3372885.3373824.

[78] Rodrigues, B., Carvalho, M., Anjos, M. F. and Sugishita, N., "Unboundedness in bilevel optimization," *Mathematical Programming* (2026). Published online 26 March 2026. https://doi.org/10.1007/s10107-026-02344-2.

[79] Cheung, K. K. H., Gleixner, A. and Steffy, D. E., "Verifying integer programming results," in *Integer Programming and Combinatorial Optimization (IPCO 2017)*, Eisenbrand, F. and Koenemann, J., eds., *Lecture Notes in Computer Science* **10328**, 148–160, Springer, Cham (2017). https://doi.org/10.1007/978-3-319-59250-3_13.

[80] Eifler, L. and Gleixner, A., "A computational status update for exact rational mixed integer programming," *Mathematical Programming* **197**(2), 793–812 (2023). https://doi.org/10.1007/s10107-021-01749-5.

[81] Bogaerts, B., Gocht, S., McCreesh, C. and Nordström, J., "Certified dominance and symmetry breaking for combinatorial optimisation," *Journal of Artificial Intelligence Research* **77**, 1539–1589 (2023). https://doi.org/10.1613/jair.1.14296.

[82] Babai, L., Simonovits, M. and Spencer, J., "Extremal subgraphs of random graphs," *Journal of Graph Theory* **14**(5), 599–622 (1990). https://doi.org/10.1002/jgt.3190140511.

[83] Brightwell, G., Panagiotou, K. and Steger, A., "Extremal subgraphs of random graphs," *Random Structures & Algorithms* **41**(2), 147–178 (2012). https://doi.org/10.1002/rsa.20413.